CSCI 566: Deep Learning and its Applications

Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California

Credits: prior USC CSCI 566 materials; Stanford CS224W.

Special thanks to Prof. Jure Leskovec for GNN materials.

USC Viterbi School of Engineering

Assignment 1 Is Released

The image-classification assignment is available on the course website.

Task	Train and evaluate an image classifier. A GPU is optional.
Submit	Upload report.pdf (at most 2 pages) and code.zip to Gradescope.
Questions	Use a new private Piazza post in the assignment folder.

Due October 26 at 11:59 PM Pacific Time.

Proposal Feedback Is Nearly Complete

The TAs are finishing the feedback for project proposals.

Current status	Most proposal feedback is complete.
Release	Watch for the TA announcement when feedback is available.
Next step	Read the comments and use them to refine your project plan.

TAs will release the feedback in the next 1–2 days.

Graph Neural Networks (GNN)

Basics

From Sequences to Connected Objects

A graph describes which objects can exchange information.

Graph representation	Choose the objects, connections and input features.
Graph neural network	Learn predictions by combining neighboring information.
Graphs of agents	Study communication paths and execution dependencies.

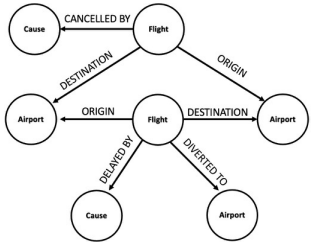
We will follow one small road network, then graphs of agents.

Course schematic. GNN sources and assumptions in speaker notes.

Why Graphs?

Graphs are a general language for describing and analyzing entities with relations/interactions

Many Types of Data are Graphs (1)



Event Graphs

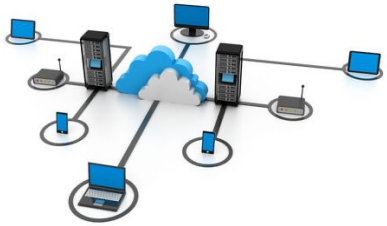
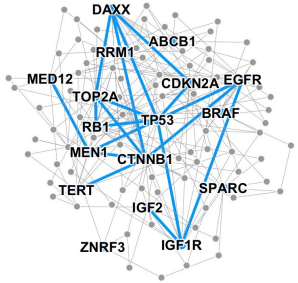


Image credit: SalientNetworks

Computer Networks



Disease Pathways

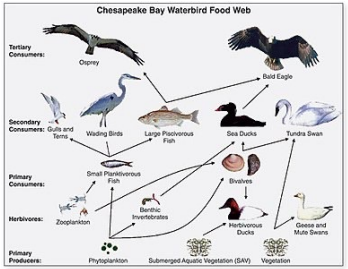


Image credit: Wikipedia

Food Webs



Image credit: Pinterest

Particle Networks

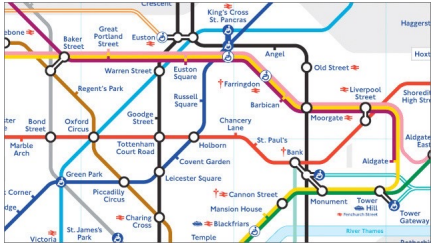


Image credit: visitlondon.com

Underground Networks

USC Viterbi School of Engineering

Many Types of Data are Graphs (2)



Image credit: Medium

Social Networks

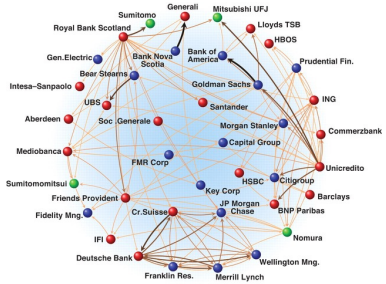


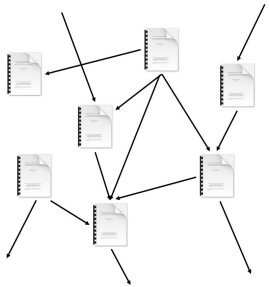
Image credit: Science

Economic Networks



Image credit: Lumen Learning

Communication Networks



Citation Networks



Image credit: Missoula Current News

Internet

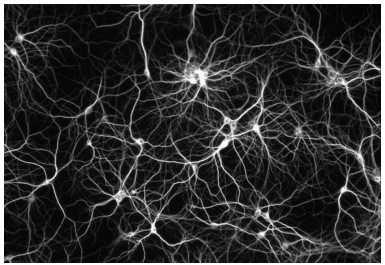


Image credit: The Conversation

Networks of Neurons

USC Viterbi School of Engineering

Many Types of Data are Graphs (3)

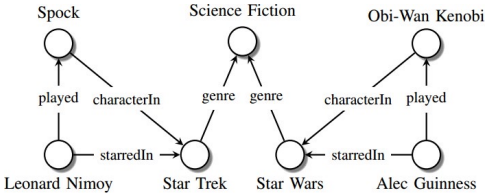


Image credit: Maximilian Nickel et al

Knowledge Graphs

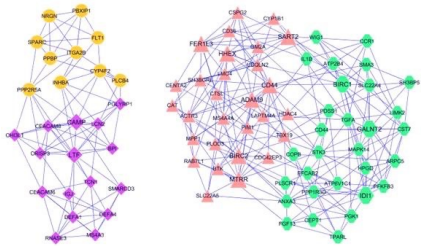


Image credit: ese.wustl.edu

Regulatory Networks

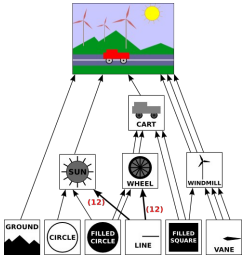


Image credit: math.hws.edu

Scene Graphs

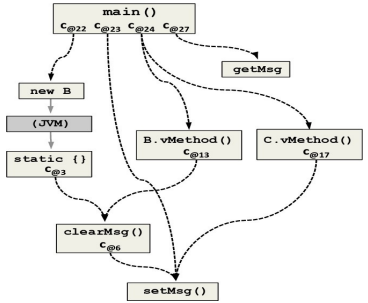
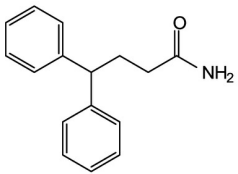


Image credit: ResearchGate

Code Graphs



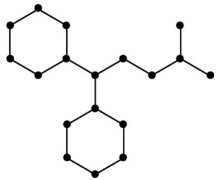


Image credit: MDPI

Molecules

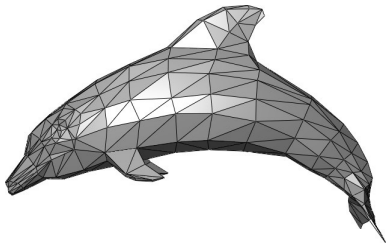


Image credit: Wikipedia

3D Shapes

USC Viterbi School of Engineering

Graphs: Machine Learning

Complex domains have a rich relational structure, which can be represented as a **relational graph**

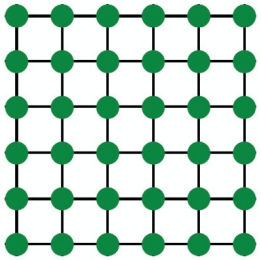
By explicitly modeling relationships, we can use this structure.

Main question:

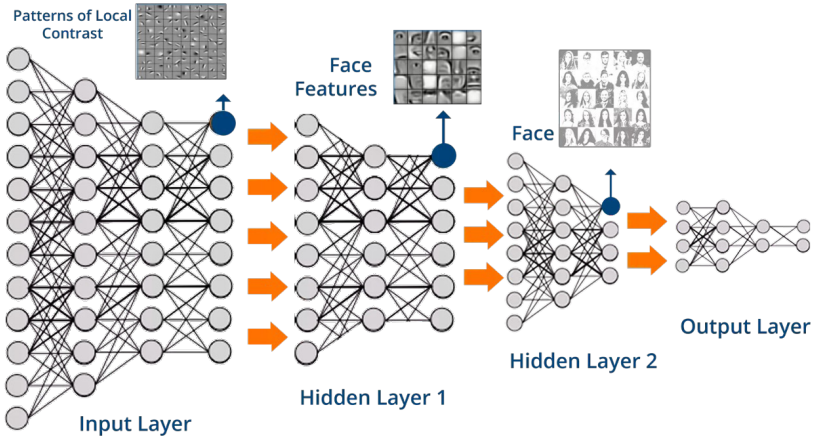
How do we take advantage of relational structure for better prediction?

Today: Modern ML Toolbox

CNN



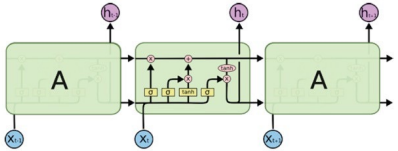
Images



Text/Speech







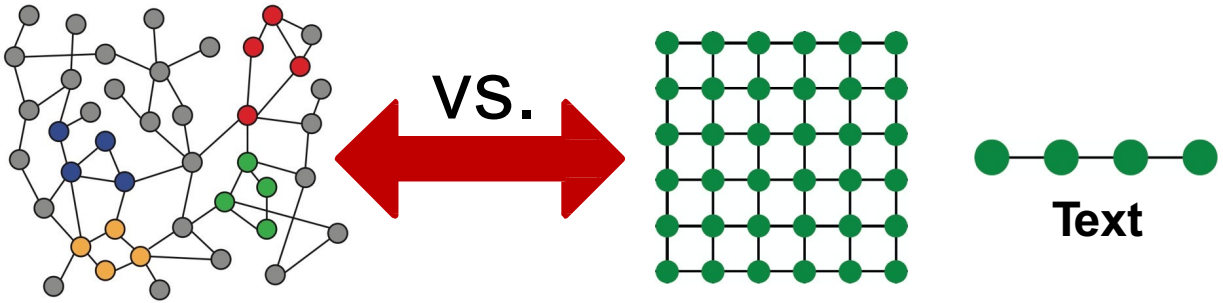
RNN/
LSTM

CNNs use grids.
Token order matters.

Why is Graph Deep Learning Hard?

Networks are complex:

Arbitrary size and complex topological structure (*i.e.*, no spatial locality like grids)

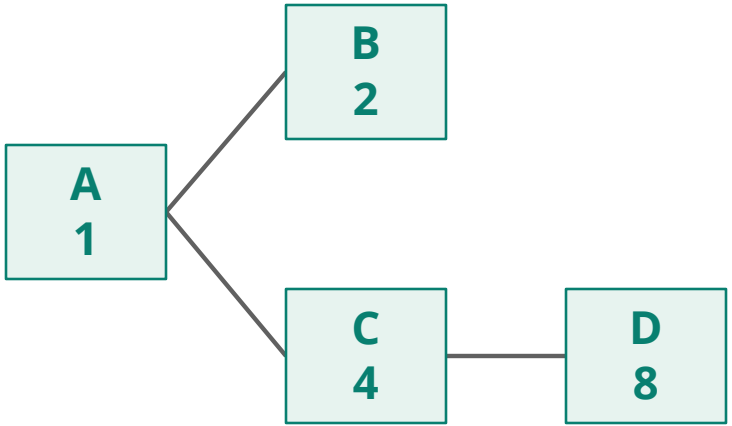


No fixed node ordering or reference point

Often dynamic and have multimodal features

Our Example: Delay Risk on Road A

We want to predict whether road A will be delayed in the next 5 minutes.



Road segments: A, B, C, D

Lines: directly connected roads

Numbers: current traffic load
The next page defines the scale.

Current traffic and connected roads are the model inputs.

Course schematic. Numbers are illustrative; sources in speaker notes.

What Do 1, 2, 4 and 8 Mean?

For this toy example: load index = vehicles counted in the last 5 minutes ÷ 10.

Road	Observed vehicles	Input load index
A	10	1
B	20	2
C	40	4
D	80	8

Index 4 means 40 observed vehicles in this example.

Course schematic. Numbers are illustrative; sources in speaker notes.

Why Is Traffic Load a Feature?

A feature is an observed value we give the model before it predicts.

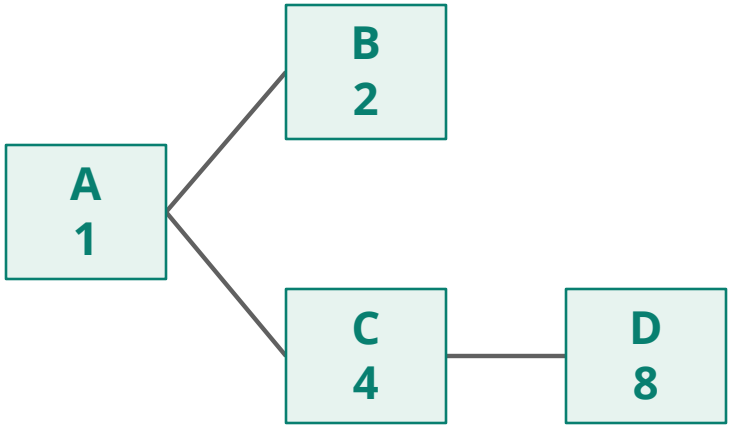
Observed now	Road A has load 1 in the current 5-minute window.
Predicted later	Will travel on A take longer than its usual travel time?
Possible extra inputs	Current speed, road capacity and time of day.

Traffic load can help predict a later delay.

Course schematic. Numbers are illustrative; sources in speaker notes.

Why Read the Neighboring Roads?

A is lightly loaded now. Connected roads B and C carry more traffic.



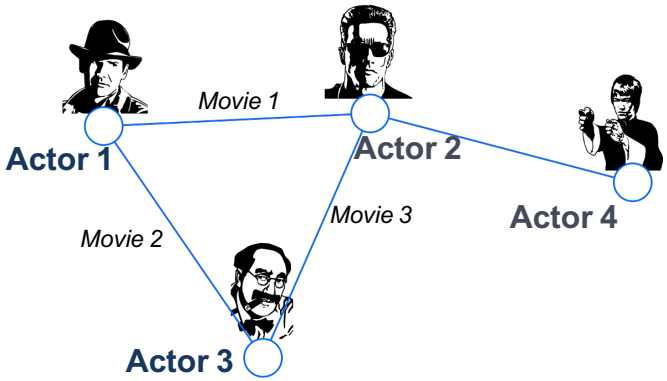
A alone sees: 1

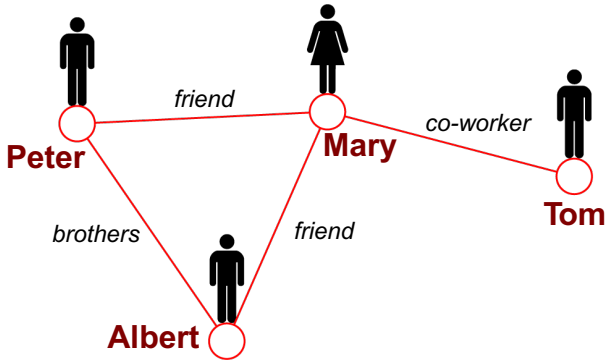
A with neighbors sees:
its own 1, plus B = 2 and C = 4

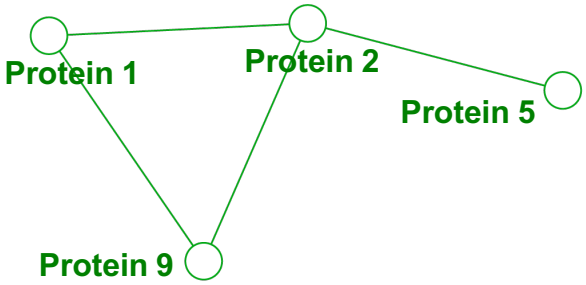
A GNN lets A use its own input and nearby inputs.

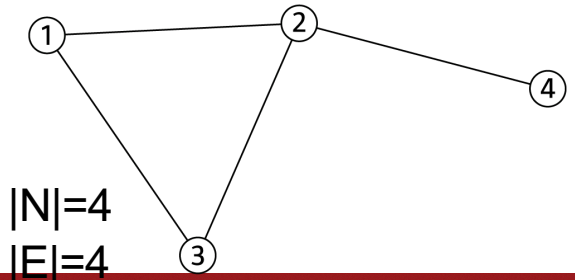
Course schematic. Numbers are illustrative; sources in speaker notes.

Graphs: A Common Language









Nodes, Edges and Features Have Different Jobs

The road graph says which road segments are connected.

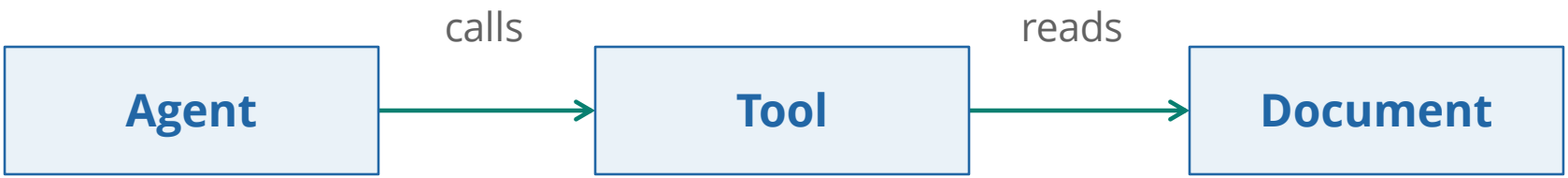
Node	One road segment, such as A.
Edge	One direct connection: A–B, A–C or C–D.
Feature	Current load: A = 1, B = 2, C = 4, D = 8.

The feature describes the road; the edge describes its connection.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Graph Can Contain Different Kinds of Objects

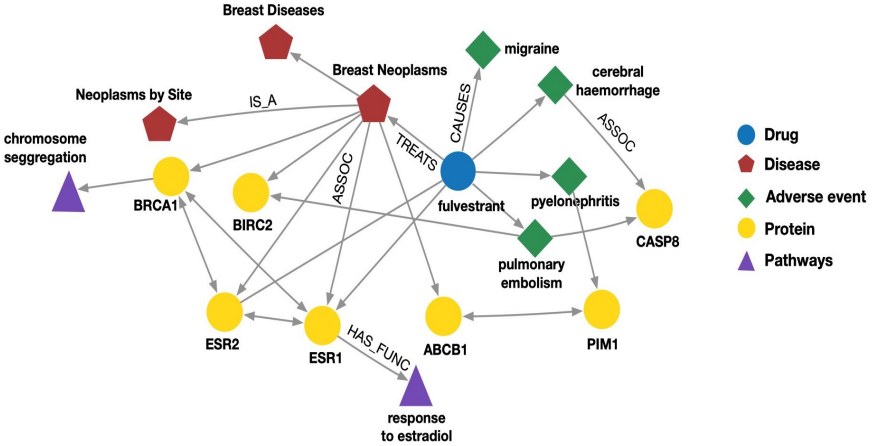
An agent, a tool and a document can be nodes in one graph.

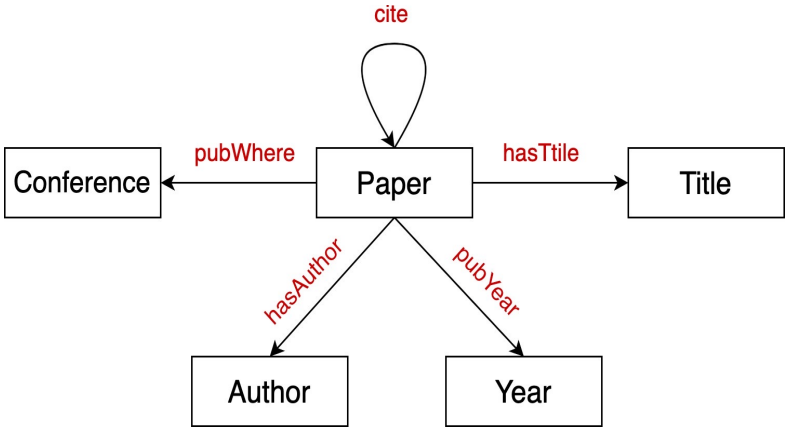


The node type and edge meaning belong to the representation.

Course schematic. GNN sources and assumptions in speaker notes.

Many Graphs are Heterogeneous





Biomedical Knowledge Graphs

Example node: Migraine
Example edge: (fulvestrant, Treats, Breast Neoplasms)
Example node type: Protein
Example edge type (relation): Causes

Academic Graphs

Example node: ICML
Example edge: (GraphSAGE, NeurIPS)
Example node type: Author
Example edge type (relation): pubYear

USC Viterbi School of Engineering

Choosing a Proper Representation

How to build a graph:

- **What are nodes?**
- **What are edges?**

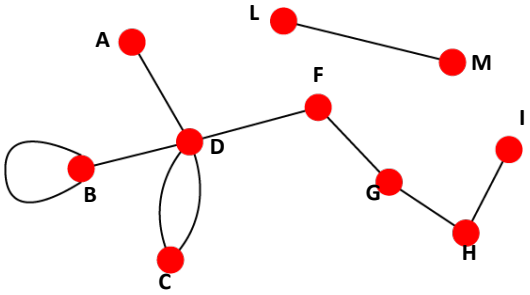
Choice of the proper network representation of a given domain/problem determines our ability to use networks successfully:

- In some cases, there is a unique, unambiguous representation
- In other cases, the representation is by no means unique
- The way you assign links will determine the nature of the question you can study

Directed vs. Undirected Graphs

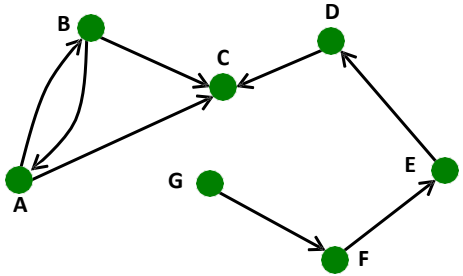
Undirected

Links: undirected (symmetrical, reciprocal)



Directed

Links: directed

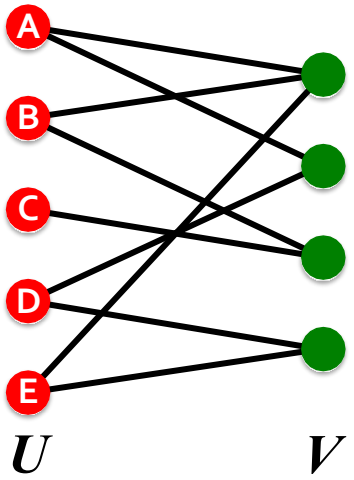


Other considerations:

- Weights
- Properties
- Types
- Attributes

Bipartite Graph

Bipartite graph is a graph whose nodes can be divided into two disjoint sets ***U*** and ***V*** such that every link connects a node in ***U*** to one in ***V***; that is, ***U*** and ***V*** are **independent sets**



Examples:

- Authors-to-Papers (they authored)
- Actors-to-Movies (they appeared in)
- Users-to-Movies (they rated)
- Recipes-to-Ingredients (they contain)

Inputs Come Before the Target

We use a current snapshot to predict what happens in the following 5 minutes.

When	What We Know	Role
Current window	Traffic loads and road links	Model inputs
Next 5 minutes	Travel time exceeds usual time? Yes / No	Training label

Training pairs the earlier inputs with the later observed label.

Course schematic. Numbers are illustrative; sources in speaker notes.

The Prediction Is Compared With a Label

For a past training example, we already know what happened in the next window.

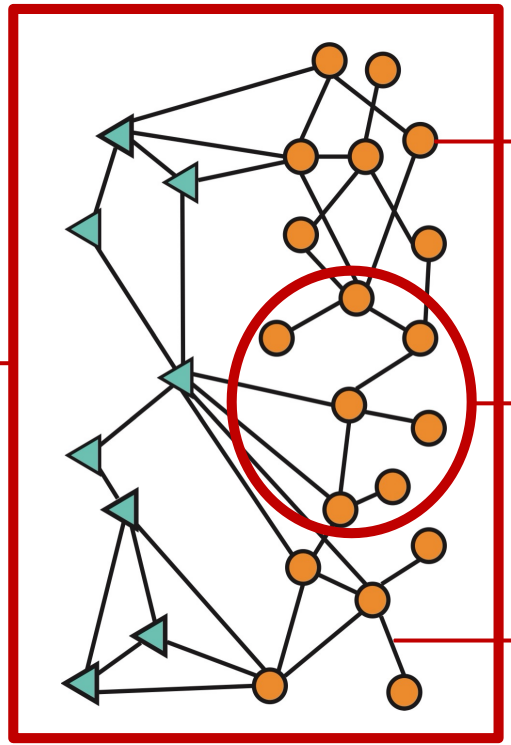


Historical Yes / No label → loss → update shared model weights

The loss trains the GNN and the prediction head.

Course schematic. Numbers are illustrative; sources in speaker notes.

Different Types of Tasks



Graph-level prediction, Graph generation

Node level

Community (subgraph) level

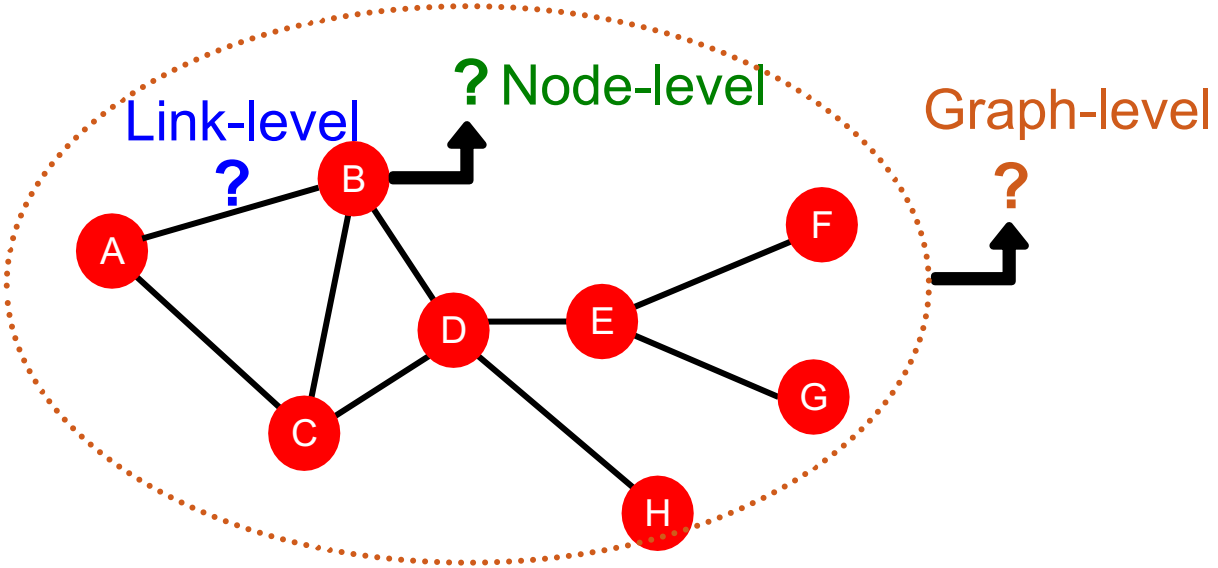
Edge-level

Machine Learning Tasks: Review

Node-level prediction

Link-level prediction

Graph-level prediction



Node, Edge and Graph Predictions

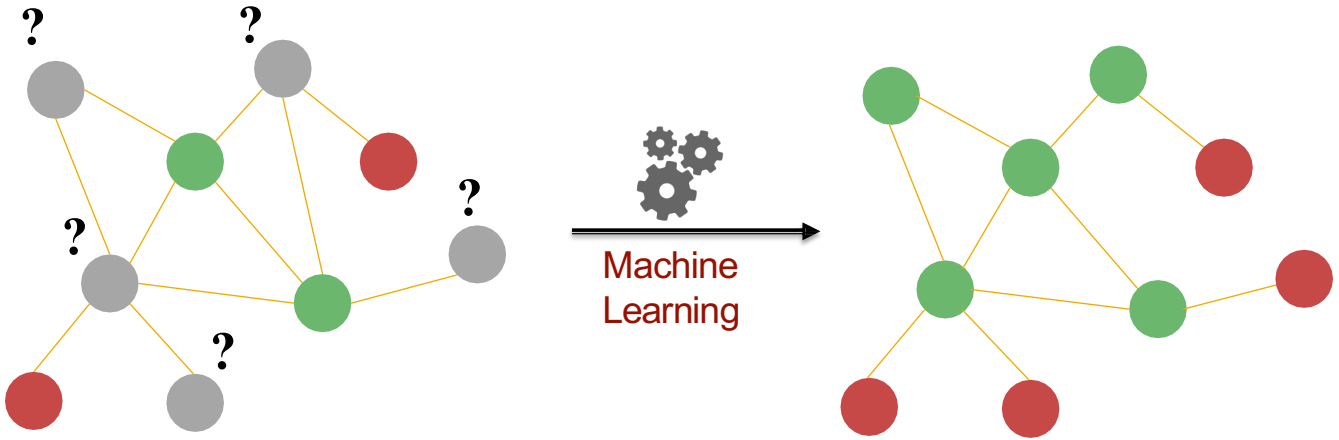
The required output tells us where to attach the prediction head.

Node output	Predict the delay risk for road A.
Pair / edge output	Predict whether two objects have a relationship.
Whole-graph output	Predict a label for the complete network.

Choose the output before choosing the model.

Course schematic. GNN sources and assumptions in speaker notes.

Node-Level Tasks

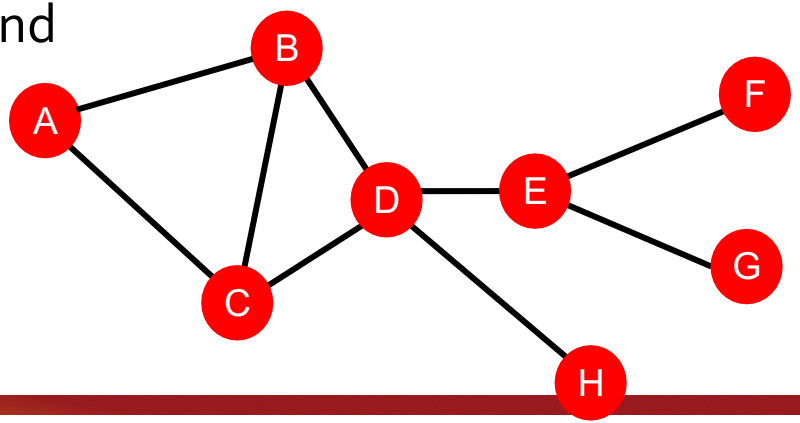


Node classification

Node-Level Network Structure

Goal: Characterize the structure and position of a node in the network:

- Node degree
- Node importance & position
  - E.g., Number of shortest paths passing through a node
  - E.g., Avg. shortest path length to other nodes
- Substructures around the node

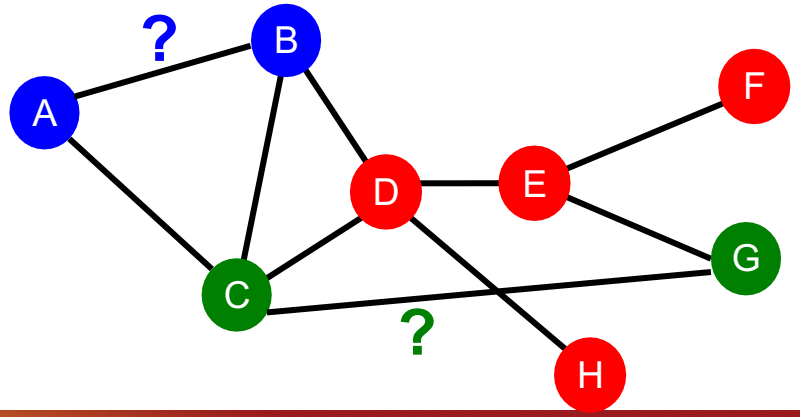


Link-Level Prediction Task

The task is to predict **new/missing/unknown links** based on the existing links.

At test time, node pairs (with no existing links) are ranked, and top K node pairs are predicted.

Task: Make a prediction for **a pair of nodes**.

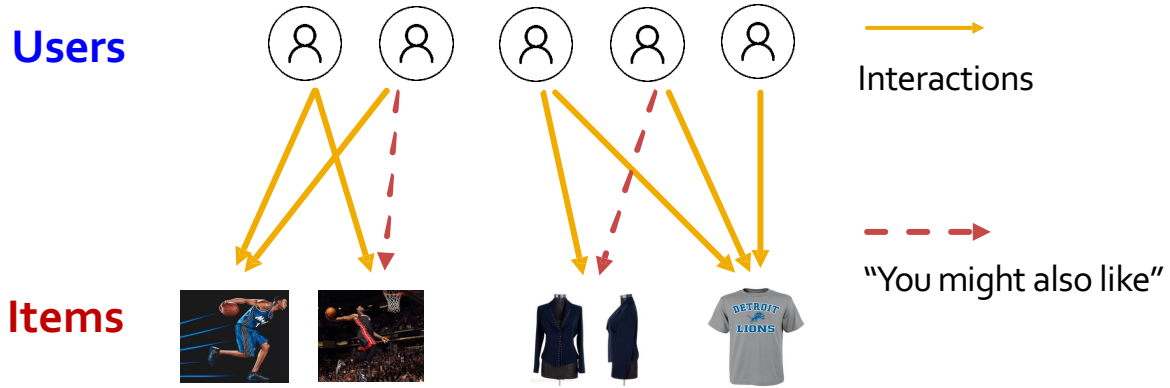


Example (1): Recommender Systems

Users interacts with items

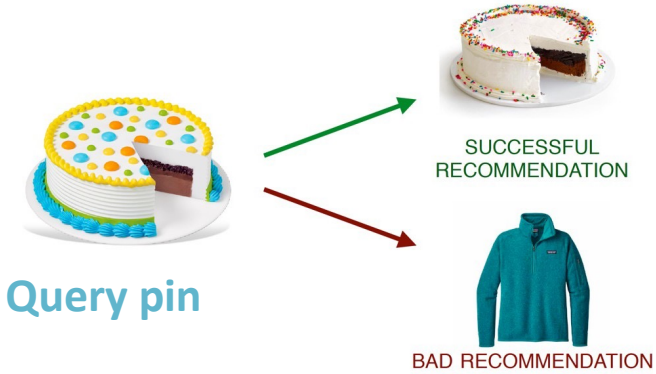
- Watch movies, buy merchandise, listen to music
- **Nodes:** Users and items
- **Edges:** User-item interactions

Goal: Recommend items users might like



PinSage: Graph-based Recommender

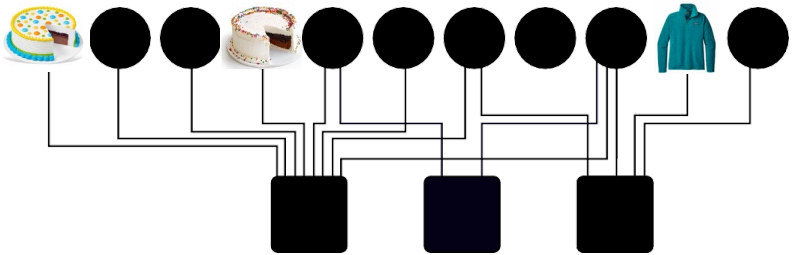
Task: Recommend related pins to users

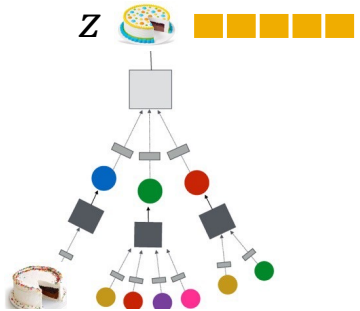


Task: Learn node embeddings z_i such that

$$d(z_{cake1}, z_{cake2}) < d(z_{cake1}, z_{sweater})$$

Predict whether two nodes in a graph are related

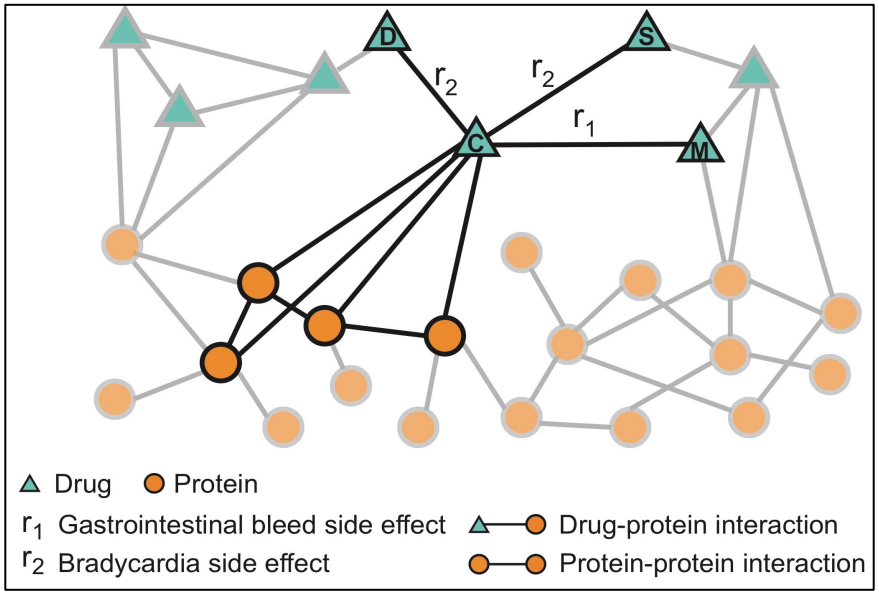




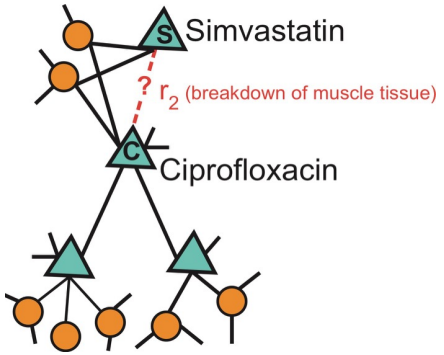
Biomedical Graph Link Prediction

Nodes: Drugs & Proteins

Edges: Interactions



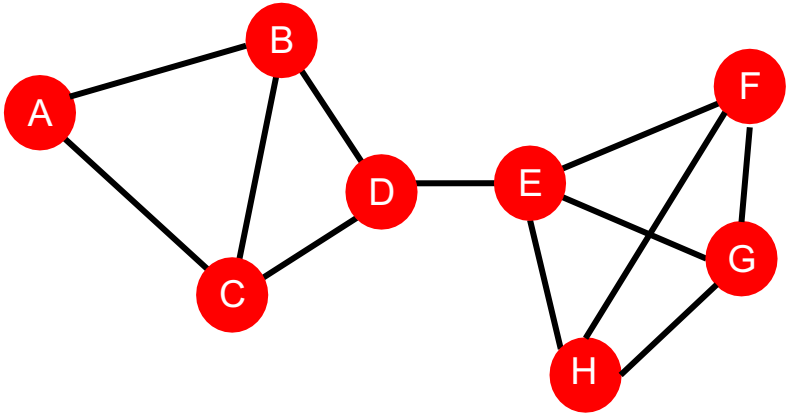
Query: How likely will Simvastatin and Ciprofloxacin, when taken together, break down muscle tissue?



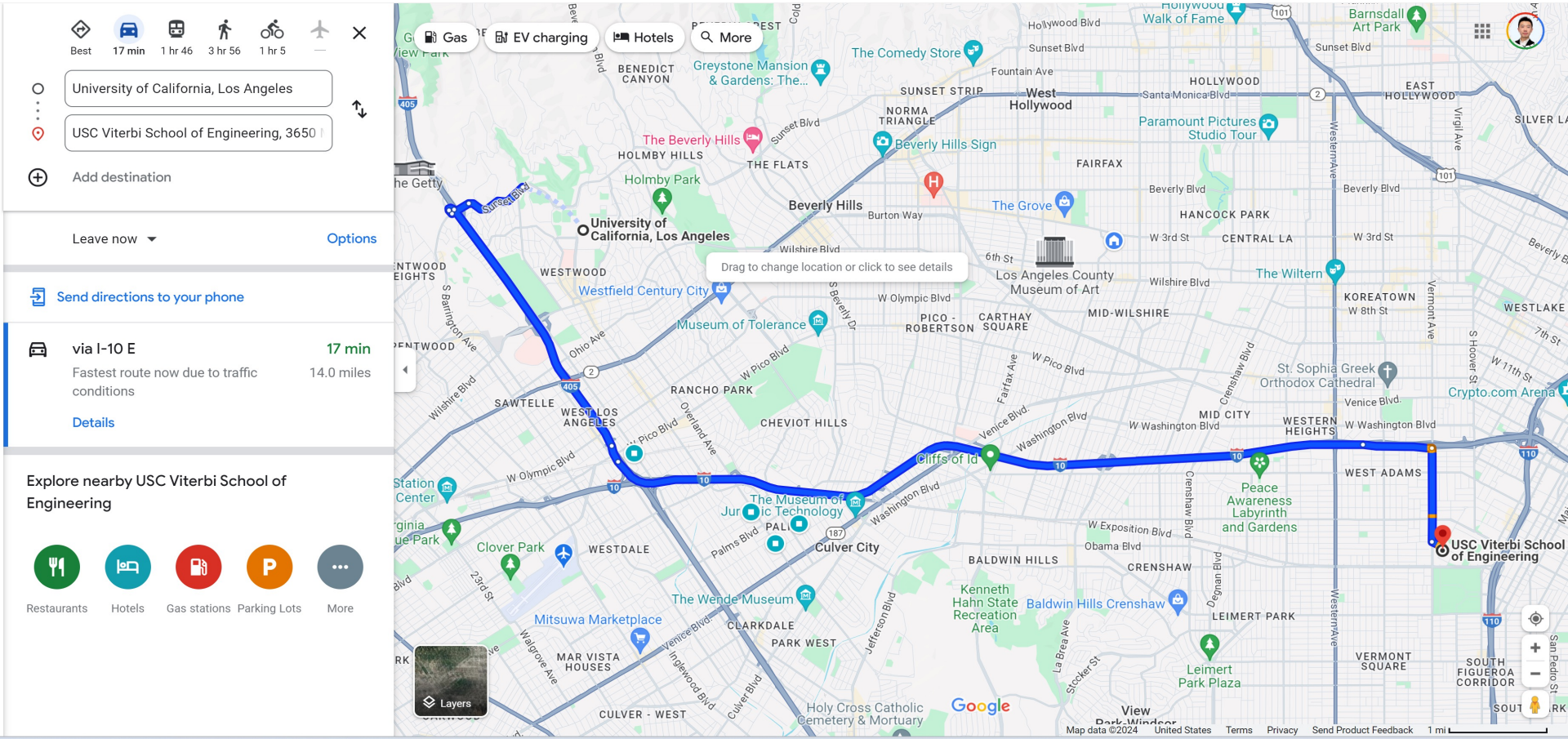
Graph-Level Features

Goal: We want make a prediction for an entire graph or a subgraph of the graph.

For example:



Example (1): Traffic Prediction

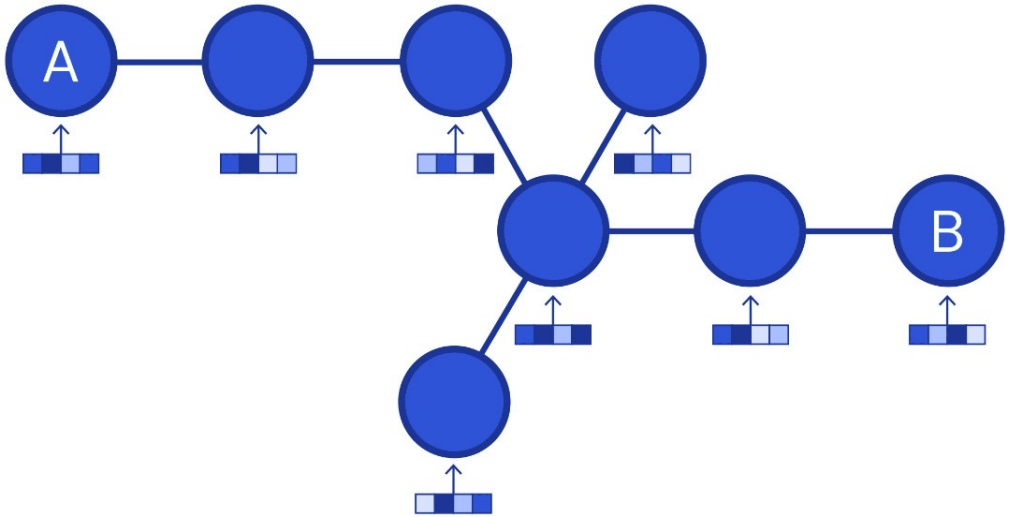


Road Network as a Graph

Nodes: Road segments

Edges: Connectivity between road segments

Prediction: Time of Arrival (ETA)



Traffic Prediction via GNN

Predicting Time of Arrival with GNNs (2020)



Used in Google Maps, 2020

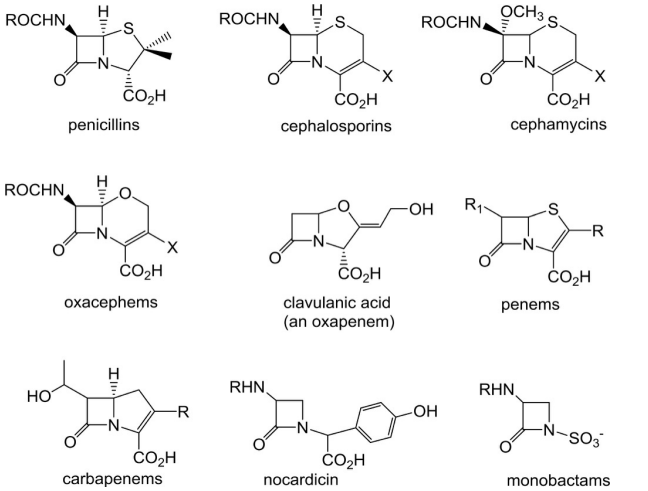
THE MODEL ARCHITECTURE FOR DETERMINING OPTIMAL ROUTES AND THEIR TRAVEL TIME.

Source: DeepMind, Sep 2020

Example (2): Drug Discovery

Antibiotics are small molecular graphs

- **Nodes:** Atoms
- **Edges:** Chemical bonds



Konaklieva, Monika I. "Molecular targets of β-lactam-based antimicrobials: beyond the usual suspects." Antibiotics 3.2 (2014): 128-142.

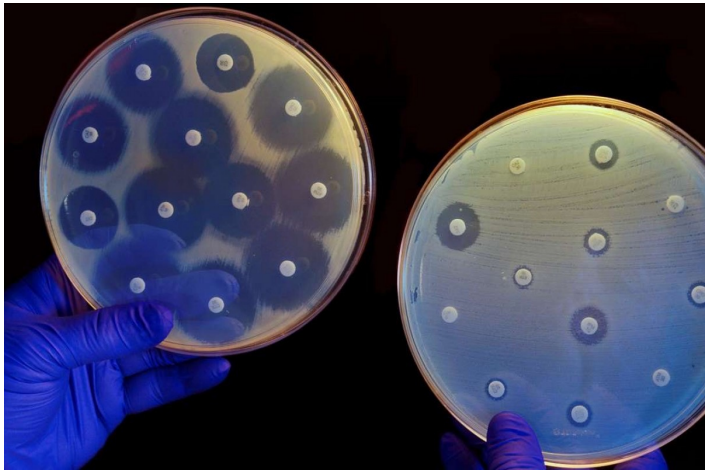
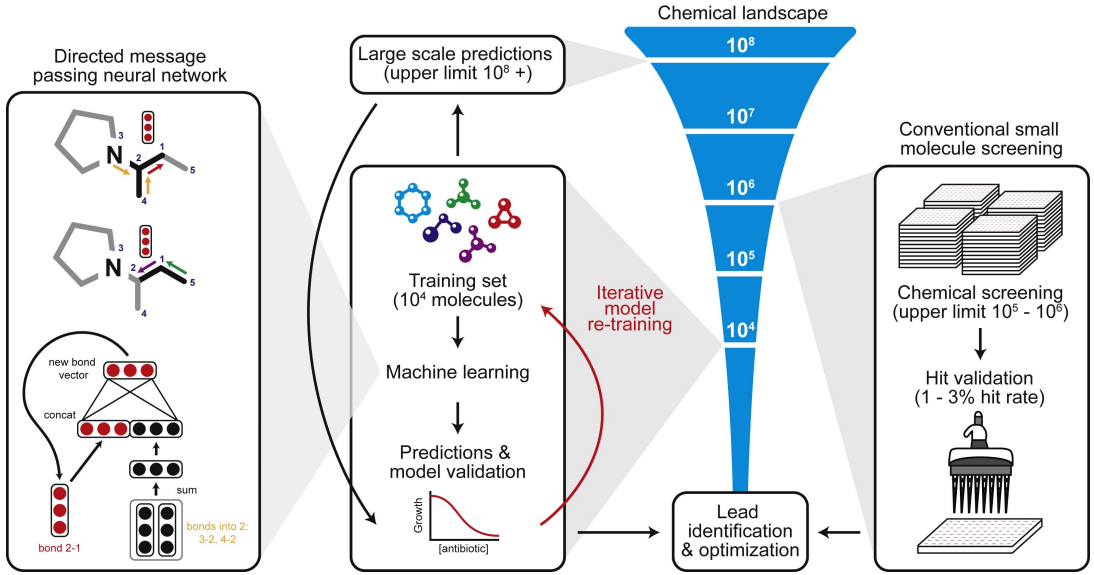


Image credit: CNN

Deep Learning for Antibiotic Discovery

A Graph Neural Network **graph classification model**
Predict promising molecules from a pool of candidates



Stokes, Jonathan M., et al. "A deep learning approach to antibiotic discovery." Cell 180.4 (2020): 688-702.

USC Viterbi School of Engineering

Graph Tasks in Drug Discovery

The molecule examples we just saw are part of a larger scientific workflow.

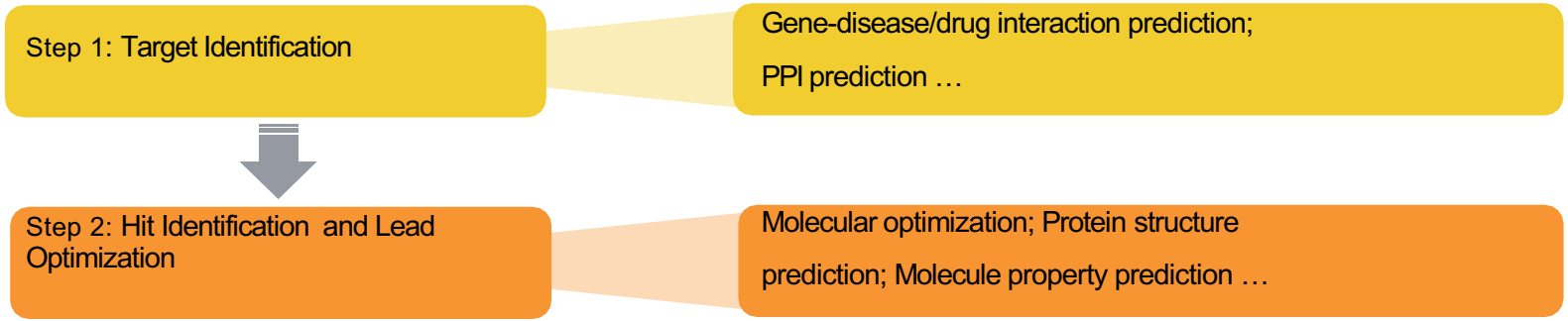
One molecule	Atoms and bonds form a graph. Predict a molecular property.
Interacting objects	Drugs and proteins form a network. Predict a possible interaction.
Scientific workflow	Therapeutics Data Commons (TDC) collects tasks and datasets for this workflow.

Choose what the graph represents and what we want to predict.

Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

Prediction Tasks in Drug Discovery

The original TDC diagram connects discovery stages with learning tasks.



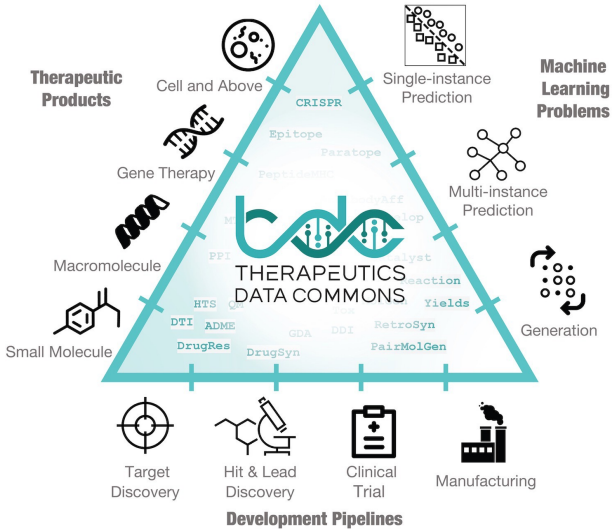
Interaction prediction: will two objects interact?
Molecule property prediction: what property does this molecule have?

Molecular properties and interactions require different outputs.

Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

TDC Organizes Objects, Tasks and Stages

Read the outer labels of the original overview. Each side answers one question.



Object

A molecule, protein or biological system.

Task

Predict one object, predict a pair, or generate an object.

Stage

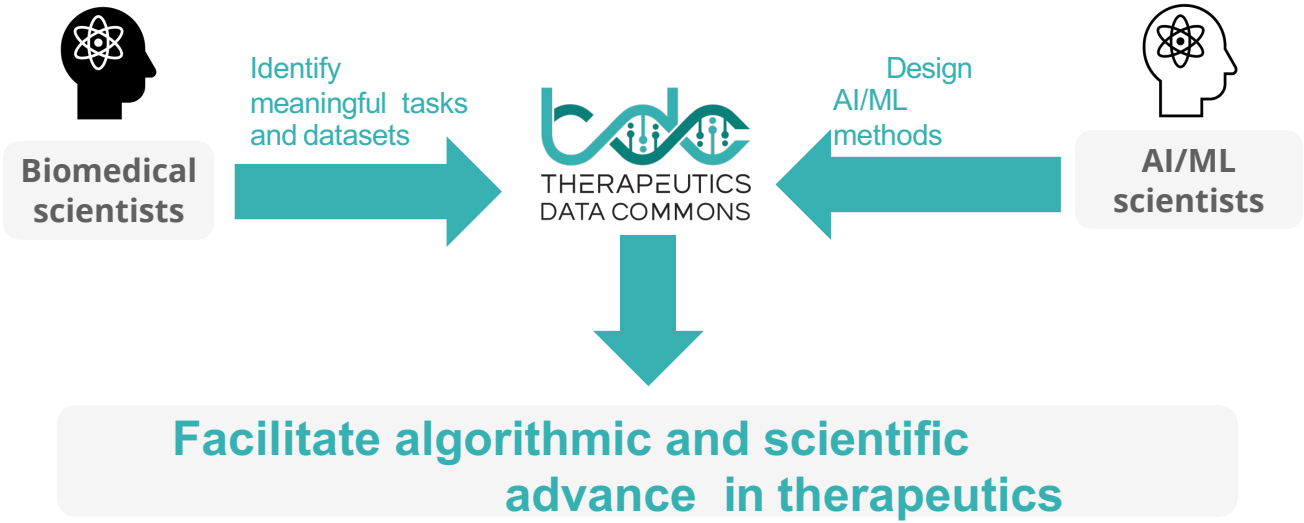
Discovery, trials or manufacturing.

Different biomedical problems require different representations.

Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

Therapeutics Data Commons

TDC provides learning tasks, datasets and benchmarks for therapeutic science.

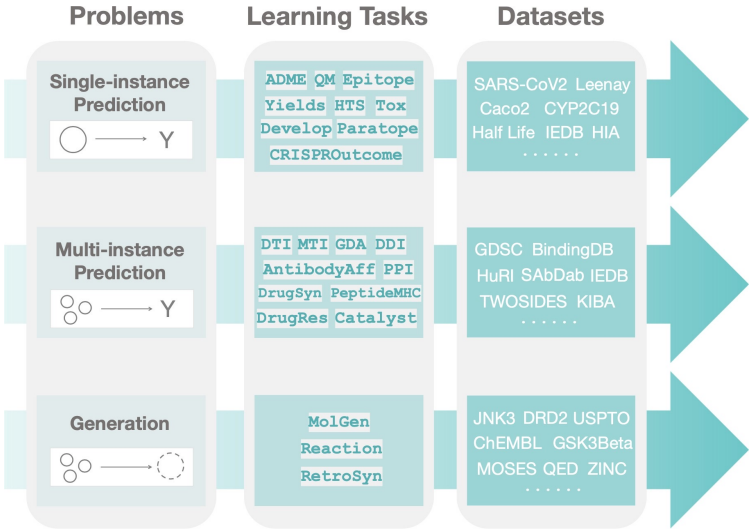


A model needs a defined task, usable inputs and evaluation data.

Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

One Object, a Pair, or a New Object

The original TDC figure groups problems by the required output.



One molecule

Predict its toxicity or another property.

A drug–protein pair

Predict whether the pair interacts.

A generated molecule

Propose an object with desired properties.

Our GNN head must match the scientific question.

Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

USC Viterbi School of Engineering

A GNN Must Produce the Right Kind of Answer

For a molecule graph, combine atom vectors. For a pair, combine two object vectors.

Molecule property	Atom vectors → pool the graph → predict one property.
Drug–protein interaction	Drug vector + protein vector → predict the interaction.
Road delay	Road A vector → prediction head → predict delay risk.

Next: how does the GNN compute these useful node vectors?

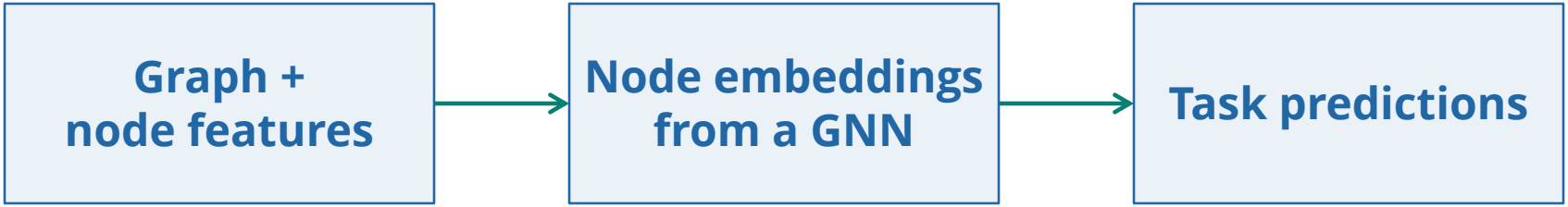
Original TDC teaching graphics; Huang et al. (2021). Sources in notes.

Graph Neural Networks (GNN)

Details

A GNN Turns Inputs Into Useful Representations

The graph gives the model structure. The task gives it a learning goal.

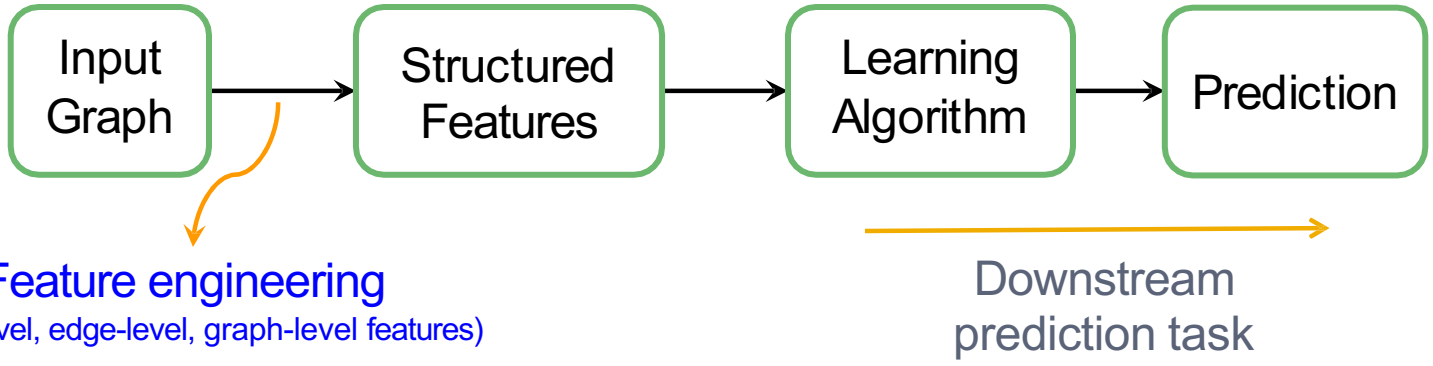


An embedding is a numeric vector used by the prediction head.

Course schematic. GNN sources and assumptions in speaker notes.

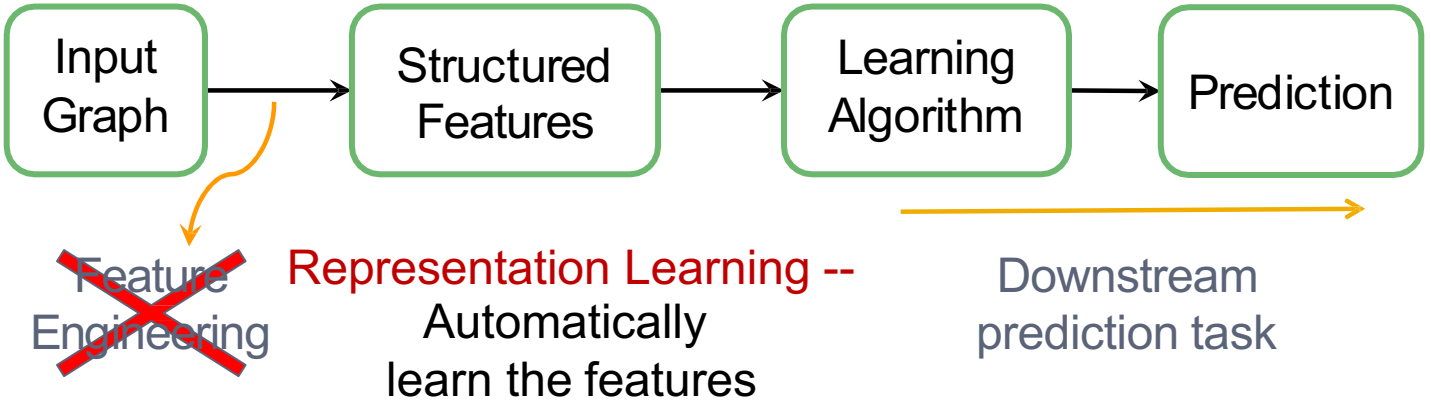
Recap: Traditional ML for Graphs

Given an input graph, extract node, link and graph-level features, then learn a model (SVM, neural network, etc.) that maps features to labels.

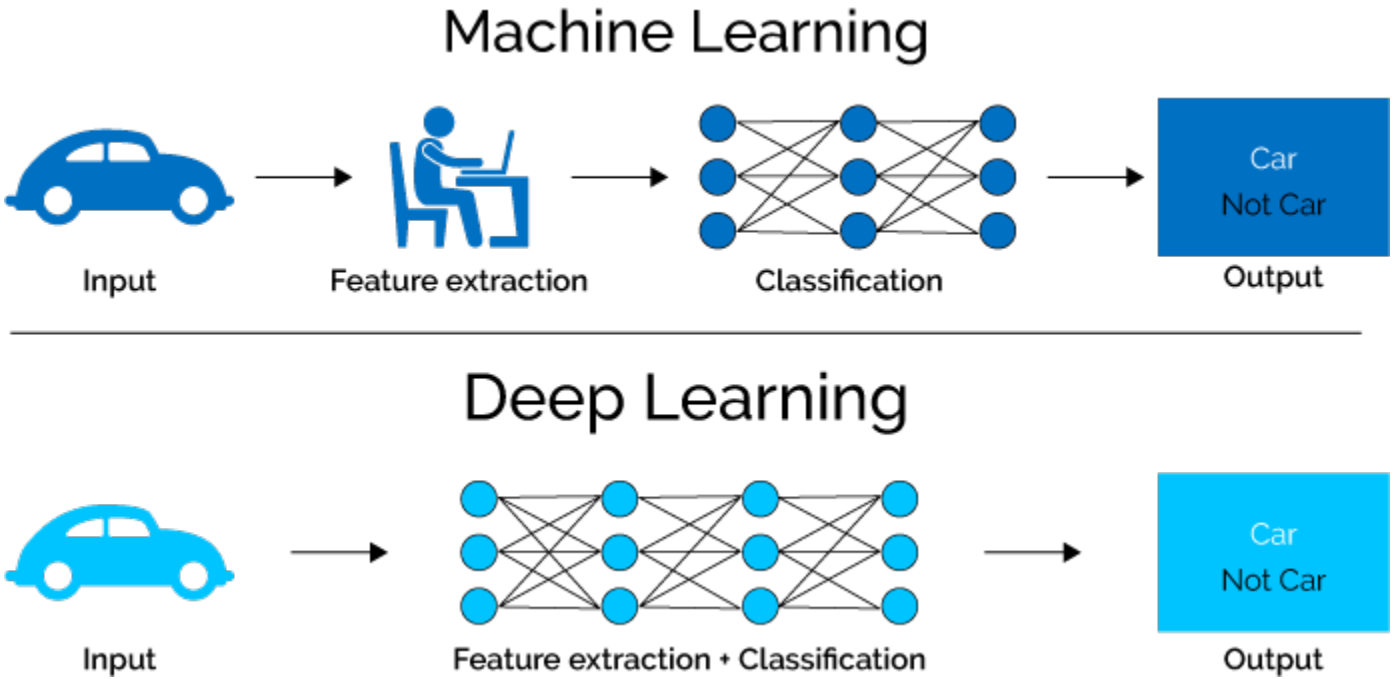


Graph Representation Learning

Graph Representation Learning alleviates the need to do feature engineering every single time.

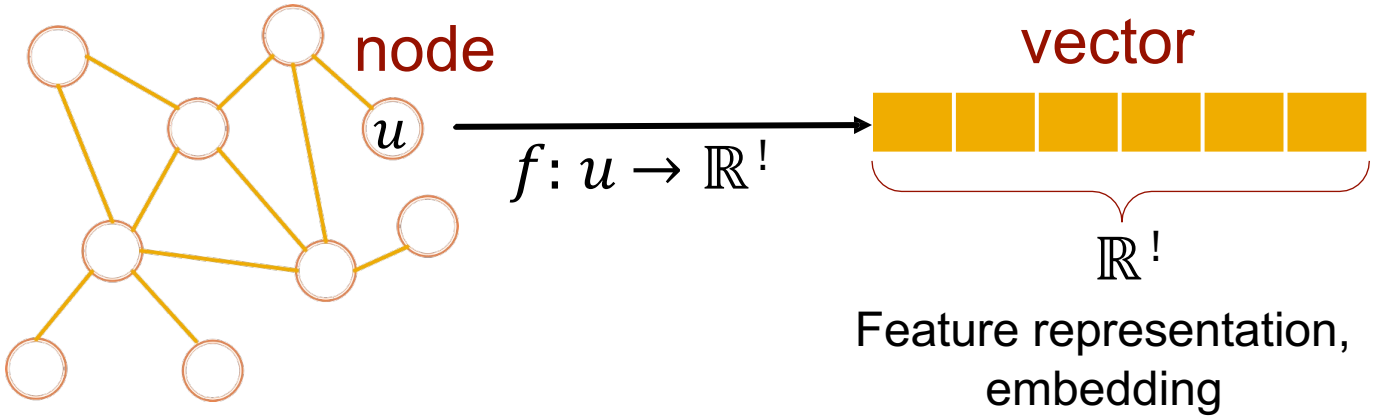


Recap: Classical ML vs. DL (Better Representations)



Graph Representation Learning

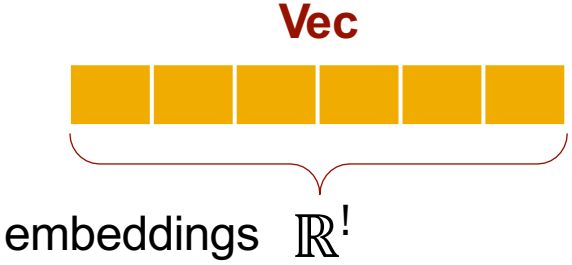
Goal: Learn node vectors useful for a chosen graph task.



Why Embedding?

Task: Map nodes into an embedding space

- The training objective determines which nodes have similar vectors. For example:
  - An edge-based objective can bring connected nodes closer.
- Encode network information
- Potentially used for many downstream predictions

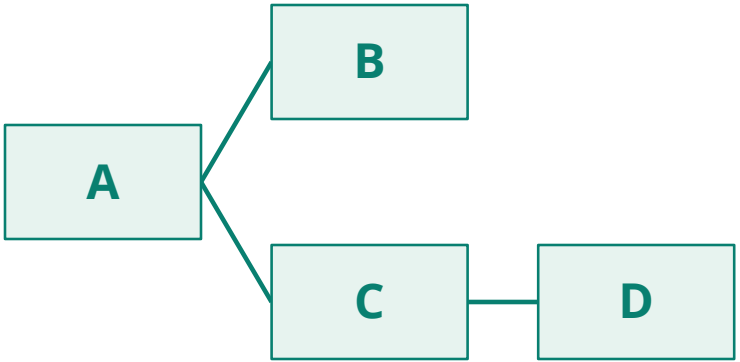


Tasks

- Node classification
- Link prediction
- Graph classification
- Anomalous node detection
- Clustering
- ….

The Adjacency Matrix Records Connections

A row tells us which neighbors one node can read.



	A	B	C	D
A	0	1	1	0
B	1	0	0	0
C	1	0	0	1
D	0	0	1	0

The matrix and the drawing describe the same graph.

Course schematic. GNN sources and assumptions in speaker notes.

Feature Rows Follow the Same Node Order

Adjacency says who is connected. X records the node inputs.

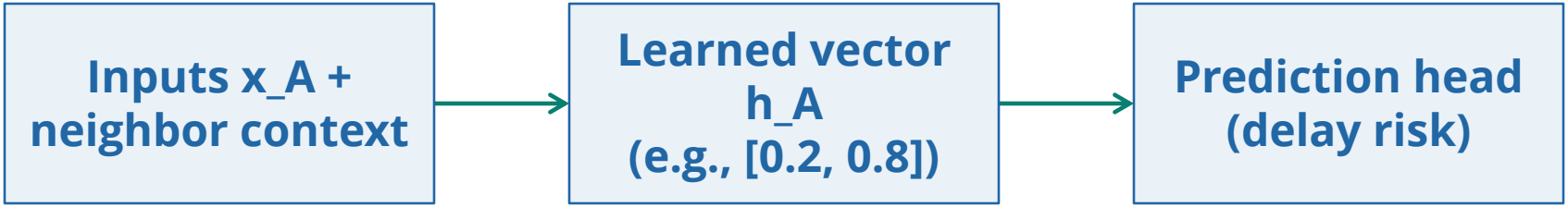
X: rows A, B, C, D	$[1, 2, 4, 8]^T$ in this simple example.
One feature here	A real road node can have several input features.
A consistent order	Reorder A and X together when renumbering nodes.

Keep node identity aligned across the graph and feature rows.

Course schematic. GNN sources and assumptions in speaker notes.

An Embedding Is a Learned Numeric Description

After neighborhood updates, each node has a vector h.



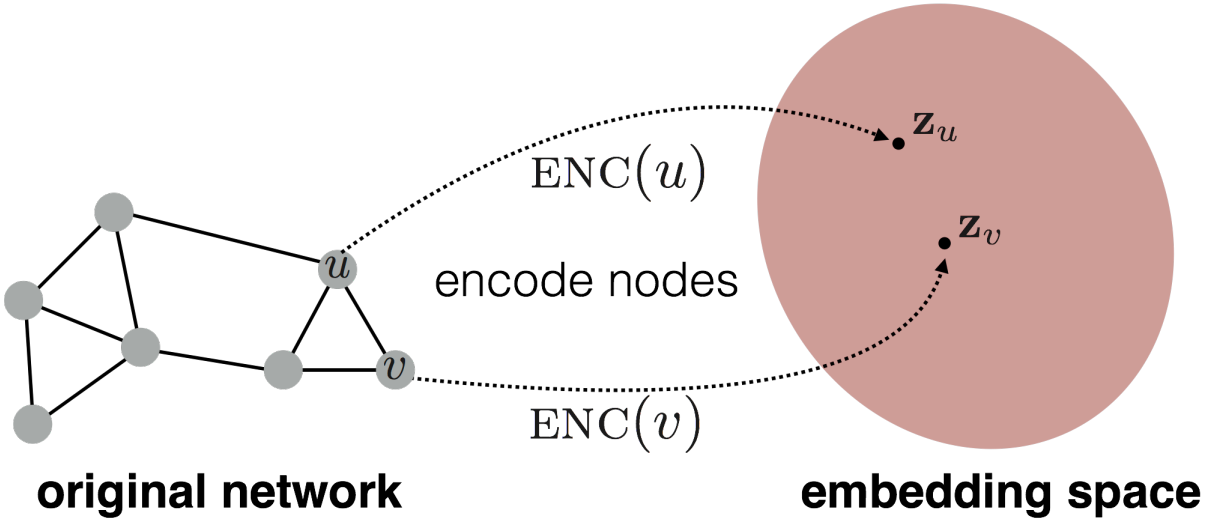
Hidden coordinates need not have a fixed human meaning.

The task trains the encoder to produce useful vectors.

Course schematic. GNN sources and assumptions in speaker notes.

A Node Becomes a Vector

The original diagram shows graph nodes mapped into a numeric space.



The training task determines what makes two vectors useful.

Course example. Sources and assumptions in speaker notes.

Two Ways to Give a Node a Vector

We need a numeric vector for each road before making a prediction.

Embedding lookup	Use a node ID to fetch its learned vector from a table.
GNN encoder	Use current features and neighbors to compute a vector.
Why compare them?	A new road has features, but may have no row in the ID table.

A lookup stores a vector. An encoder computes a vector.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Lookup Needs an Entry for Each ID

Suppose training has learned one vector for each of roads A, B, C and D.

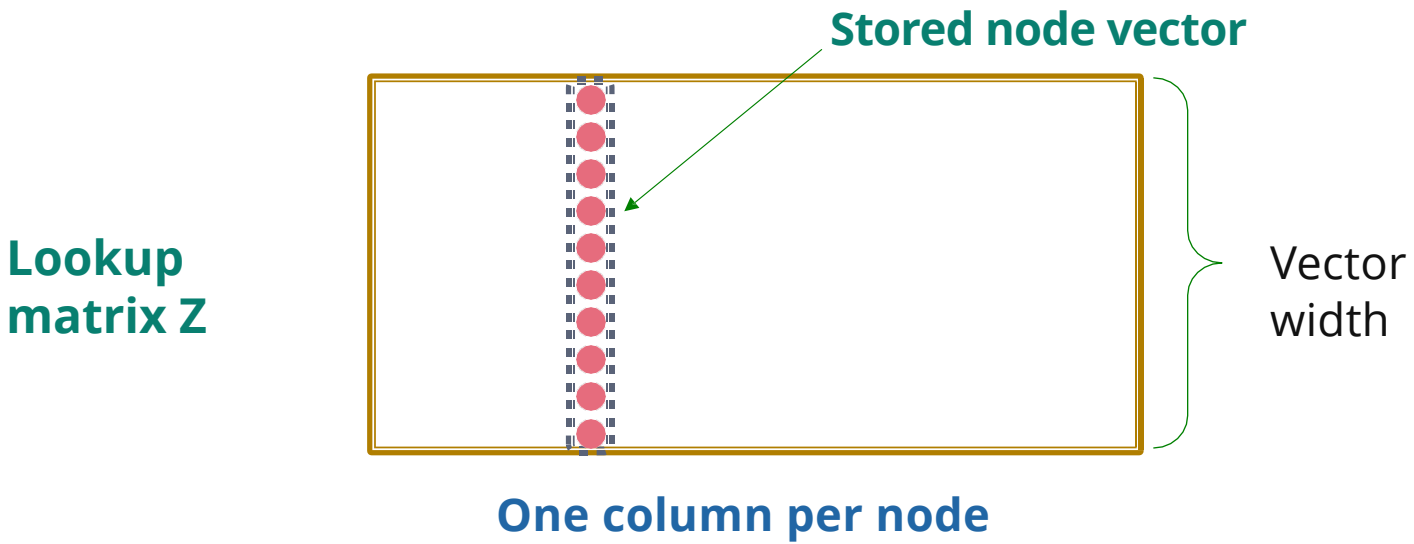
Node ID	Stored Learned Vector
A	[0.2, 0.8]
B	[0.5, 0.1]
E: new road	No learned entry yet

A newly added road E has no trained table entry yet.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Lookup Stores One Vector per Node

In this illustration, one column stores the vector for one node ID.



A lookup needs an entry for every node ID.

Course example. Sources and assumptions in speaker notes.

A GNN Reuses a Computation Rule

The learned object is a function with shared weights, used at every road.

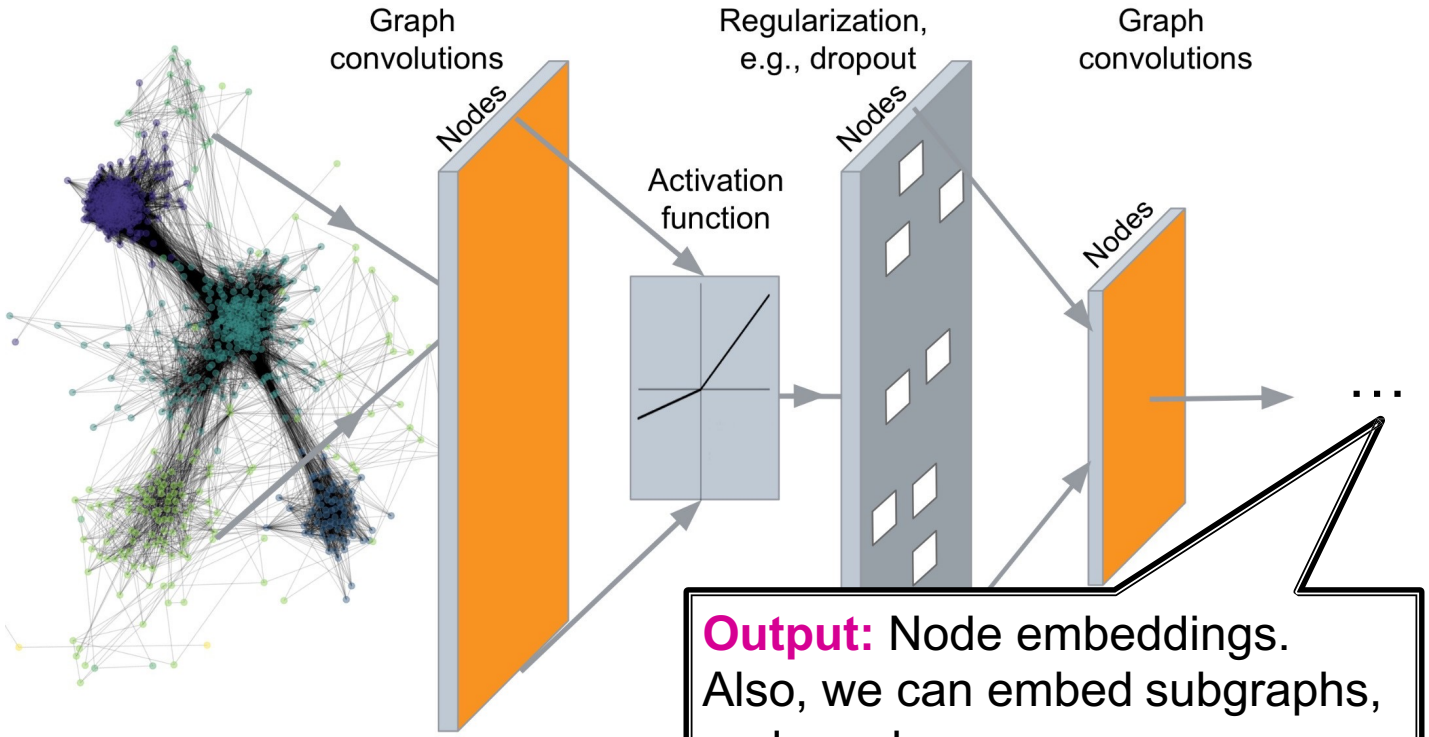


The inputs change from road to road. The learned weights are shared.

Road E can run that function using its features and neighbors.

Course schematic. Numbers are illustrative; sources in speaker notes.

Deep Graph Encoders



Output: Node embeddings.
Also, we can embed subgraphs, and graphs

USC Viterbi School of Engineering

Setup

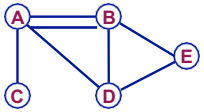
Assume we have a graph G:

- V is the **vertex set**
- $\boldsymbol{A}$ is the **adjacency matrix** (assume binary)
- $\boldsymbol{X} \in \mathbb{R}^{|V| \times m}$ is a matrix of **node features**
- v: a node in V; $N(v)$: the set of neighbors of v.
- **Node features:**
  - Social networks: User profile, User image
  - Biological networks: Gene expression profiles, gene functional information
  - When there is no node feature in the graph dataset:
    - Indicator vectors (one-hot encoding of a node)
    - Vector of constant 1: [1, 1, …, 1]

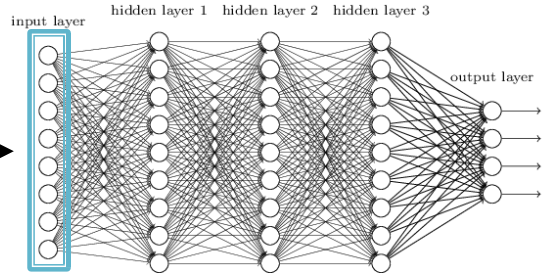
A Naïve Approach

Join adjacency matrix and features

Feed them into a deep neural net:



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



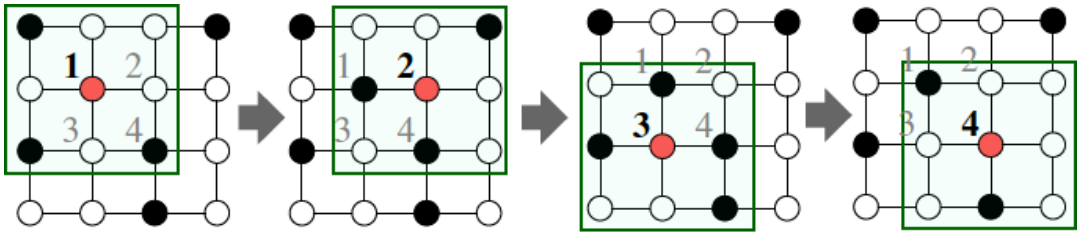
?

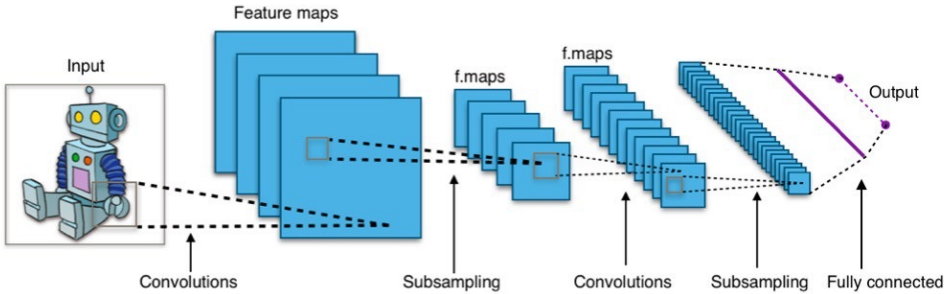
Issues with this idea:

- Input width grows with graph size
- Not applicable to graphs of different sizes
- Sensitive to node ordering

Idea: Convolutional Networks

CNN on an image:



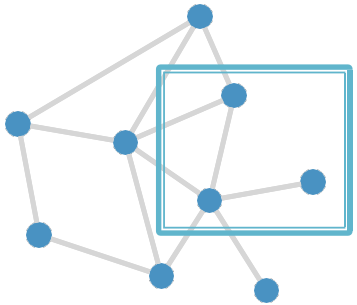


Goal is to generalize convolutions beyond simple lattices

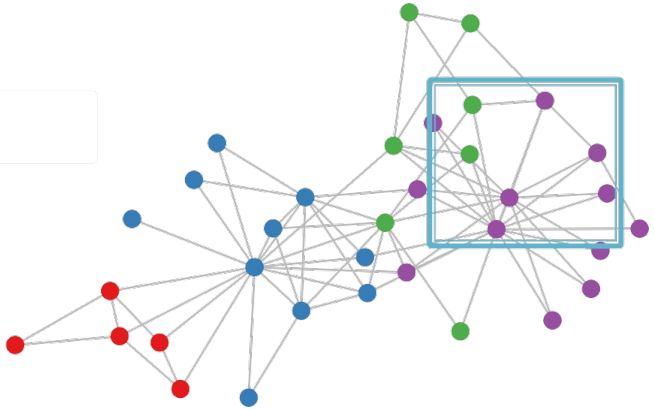
Leverage node features/attributes (e.g., text, images)

Real-World Graphs

But our graphs look like this:



or this:

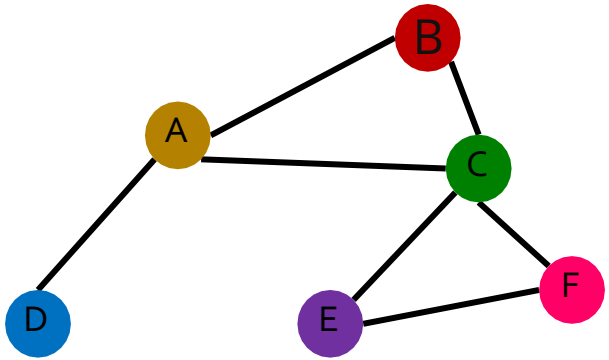


- Graphs have neighborhoods, but no fixed grid-shaped window.
- Node numbering is arbitrary

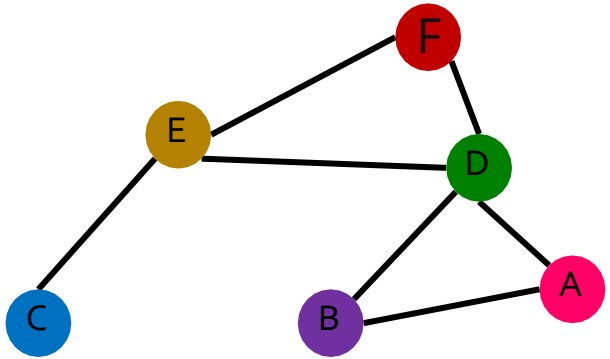
One Graph, Two Node Numberings

In this six-node sketch, colors track the objects. Letters are temporary storage labels.

Order 1



Order 2



Example: the gold object changes label A → E, while its links stay intact.

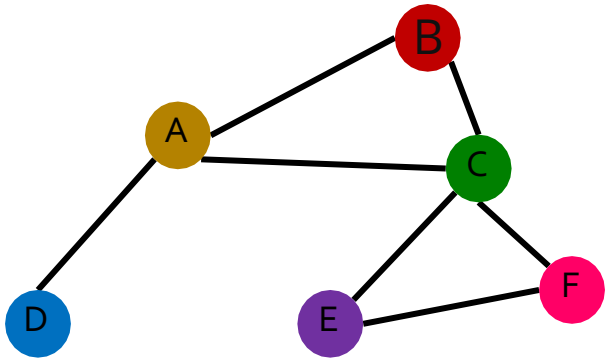
The same colored node keeps the same neighbors.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Invariance: One Answer for the Whole Graph

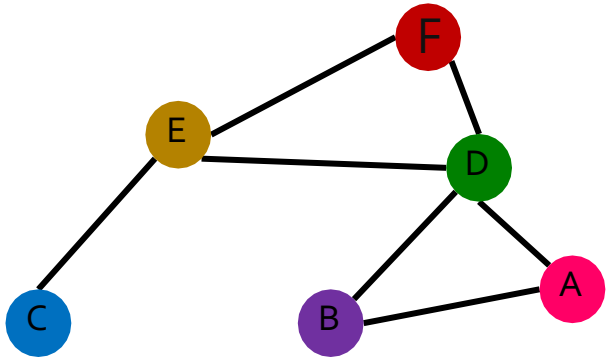
Suppose our task asks whether this entire graph is connected.

Order 1



Connected: Yes

Order 2



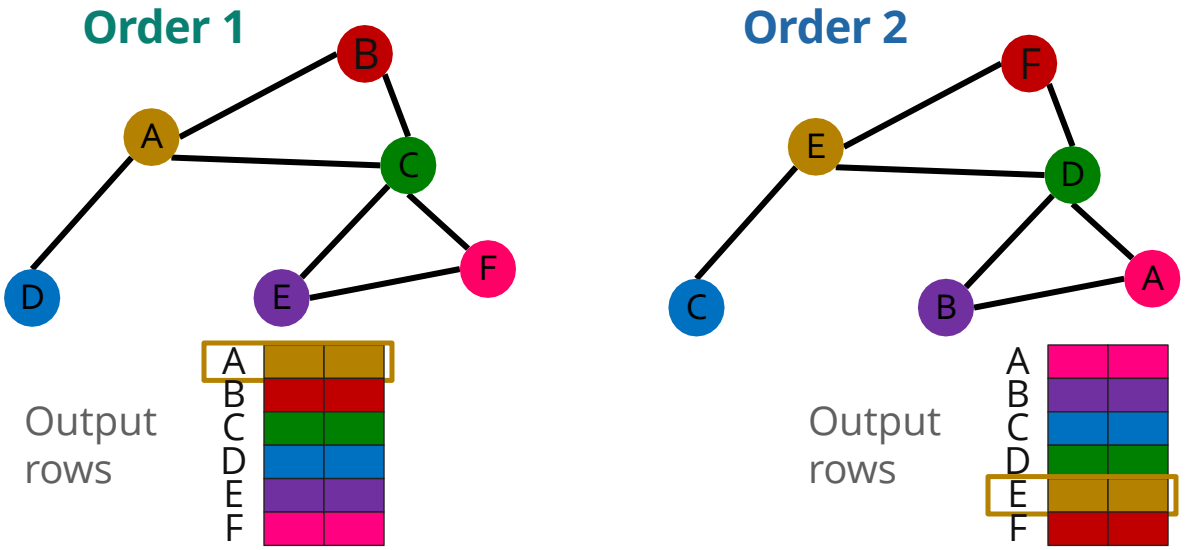
Connected: Yes

Renumbering leaves the whole-graph answer unchanged.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Equivariance: One Vector for Each Node

The colored rows below belong to the colored nodes above.

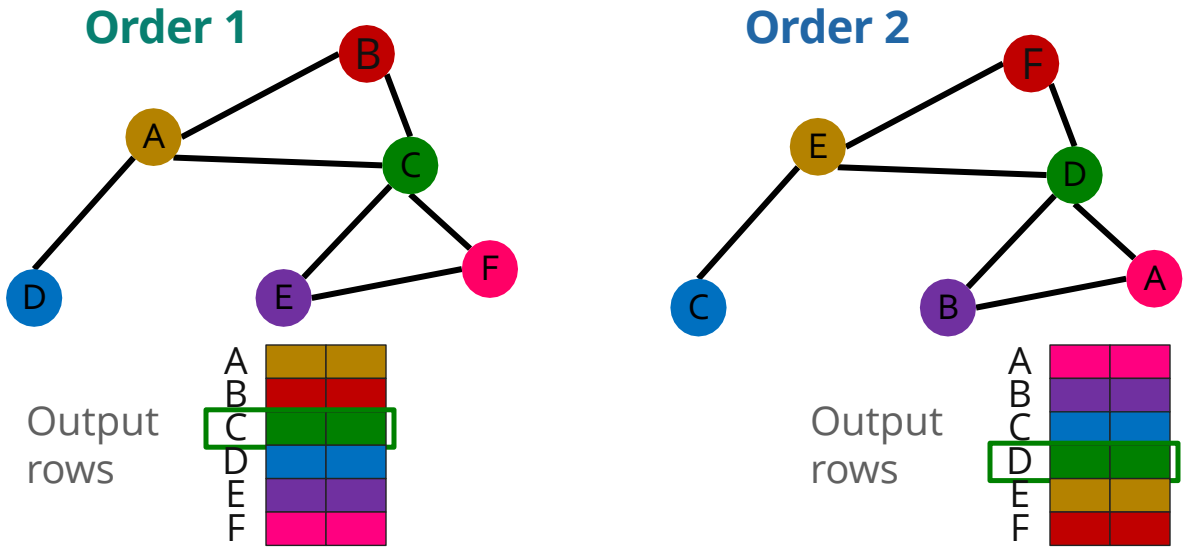


The gold object keeps its vector, moving from row A to row E.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

The Green Object Moves From Row C to Row D

Follow a second node through the same renumbering. Its vector moves with its ID.



Equivariance keeps every node's output attached to that node.

Original Stanford CS224W colored-node diagrams, with introductory annotations.

Two Output Types Need Two Properties

What should happen when we reorder the input rows?

One output per road	Output rows follow the same reordering: equivariance.
One output per graph	The graph output stays the same: invariance.
Same graph throughout	We changed its storage order, keeping its connections intact.

The required property depends on what the model outputs.

Course schematic. Numbers are illustrative; sources in speaker notes.

Different Slot Weights Can Change a Graph Score

Suppose a model treats the first file position differently from the second.

Toy graph score = 1 × first slot + 2 × second slot

File Order	Stored Values	Computed Score
A, B	[1, 2]	1×1 + 2×2 = 5
B, A	[2, 1]	1×2 + 2×1 = 4

Changing file order changes this score for the same two roads.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Shared Node MLP Follows the Roads

An MLP is a small feedforward neural network. Use the same one for every node.

Simple illustration of a shared function: f(x) = 2x

File Order	Input Rows	Output Rows
A, B	[1, 2]	[2, 4]
B, A	[2, 1]	[4, 2]

A gets the same result wherever its row is stored.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Shared Rule Can Also Read Neighbors

Back to our road example: A is the named road with load 1.

Shared node MLP	Apply the same function to each road's own features.
GNN at road A	Read A = 1 and neighbor inputs B = 2, C = 4.
Neighbor order	The mean of 2 and 4 is 3 in either listing order.

One shared update combines each node with its neighbors.

Course example. Sources and assumptions in speaker notes.

A Message Brings a Neighbor Into the Update

Road A reads information from B and C, then updates its vector.



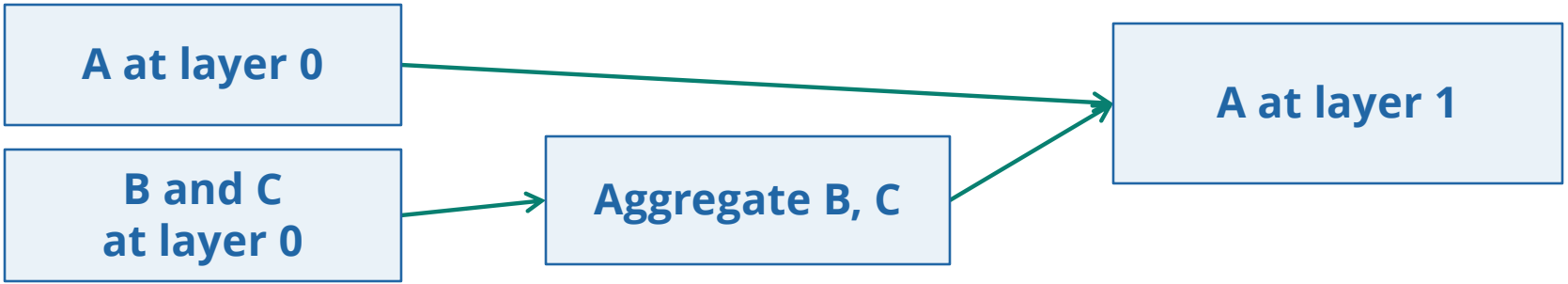
Each node combines its own information with its neighbors.

Course schematic. GNN sources and assumptions in speaker notes.

Input Graph and Computation Graph

The road graph describes connections. The computation graph describes one update.

Input graph: A is connected to B and C.

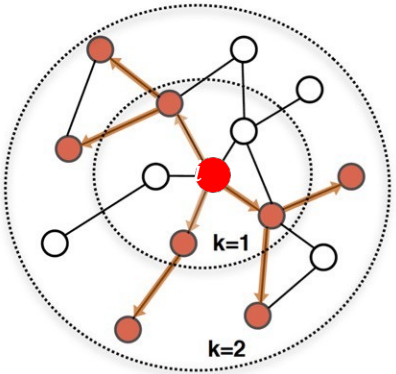


Repeated labels refer to the same road at different layers.

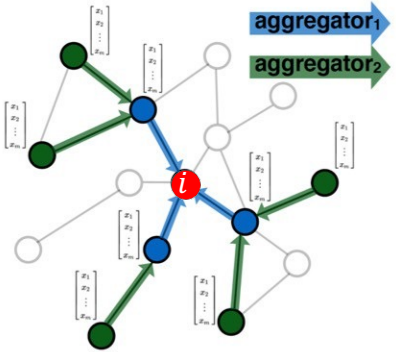
Course schematic. Sources and assumptions in speaker notes.

Graph Convolutional Networks

Idea: Node's neighborhood defines a computation graph



Determine node computation graph

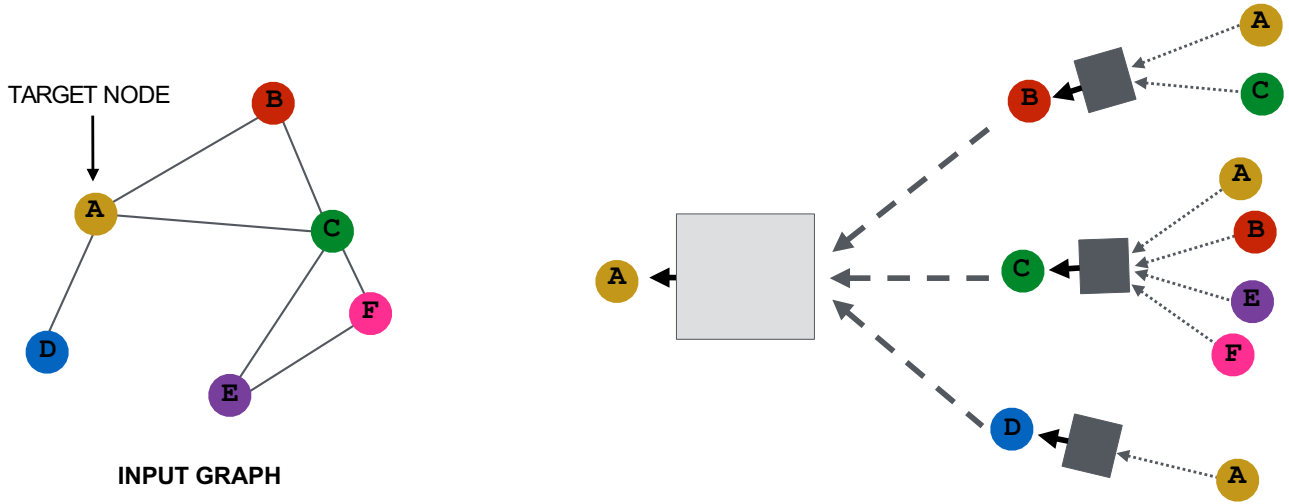


Propagate and transform information

Learn how to propagate information across the graph to compute node features

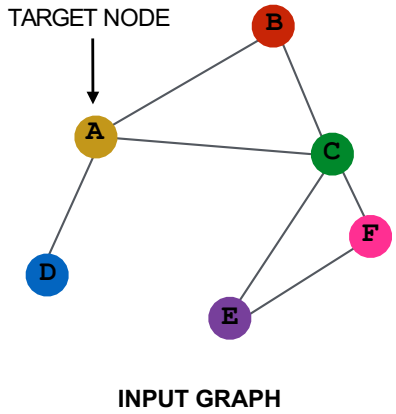
Idea: Aggregate Neighbors

Key idea: Generate node embeddings based on **local network neighborhoods**

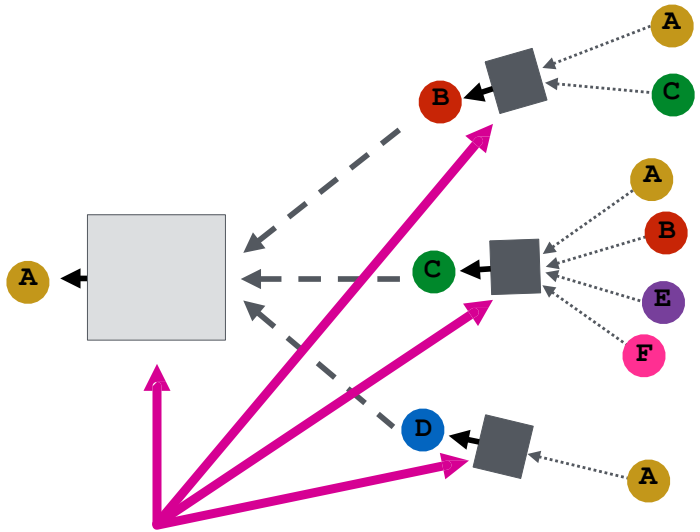


Idea: Aggregate Neighbors

Intuition: Nodes aggregate information from their neighbors using neural networks



INPUT GRAPH

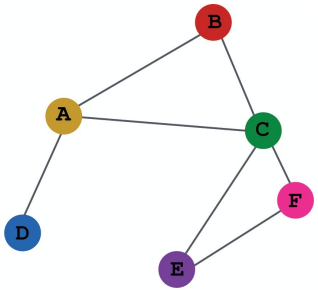


Neural networks

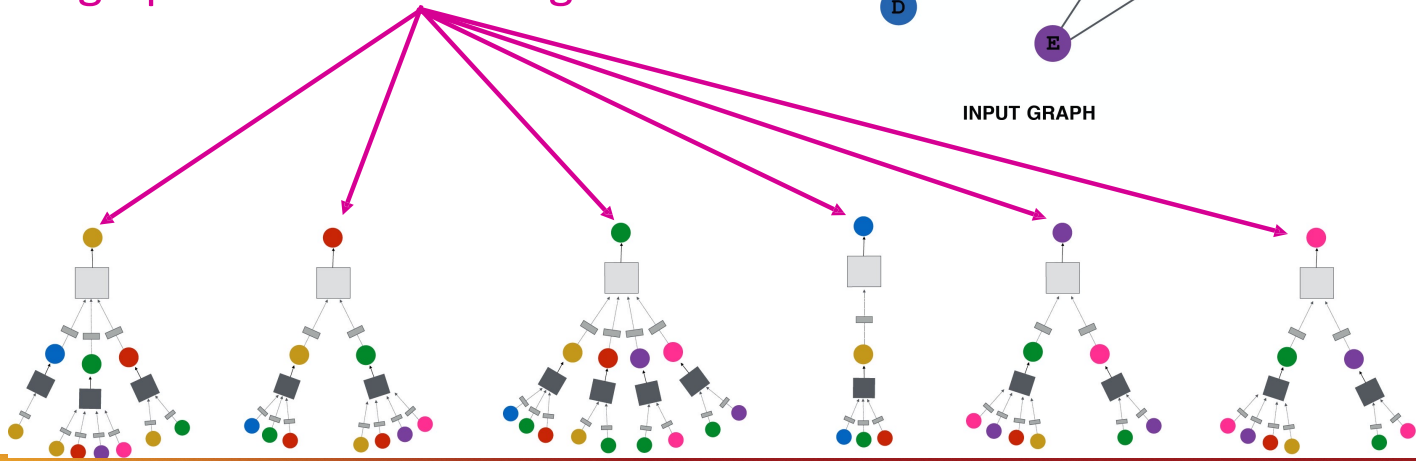
Idea: Aggregate Neighbors

Intuition: Network neighborhood defines a computation graph

Every node defines a computation graph based on its neighborhood!



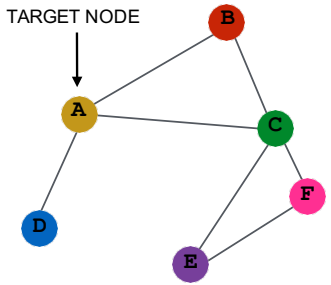
INPUT GRAPH



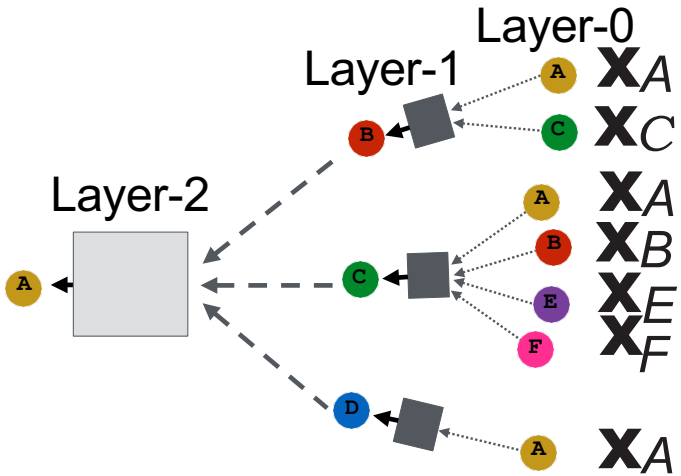
Deep Model: Many Layers

Model can be of arbitrary depth:

- Nodes have embeddings at each layer
- Layer-0 embedding of node v is its input feature, x_v
- Layer-k embeddings can use information from up to k hops away.

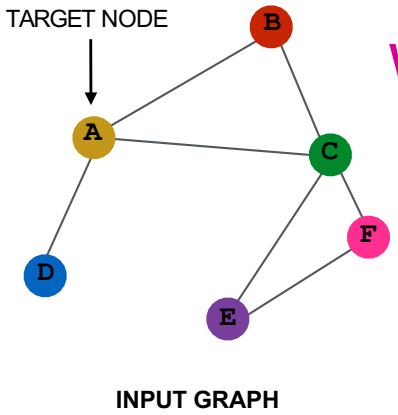


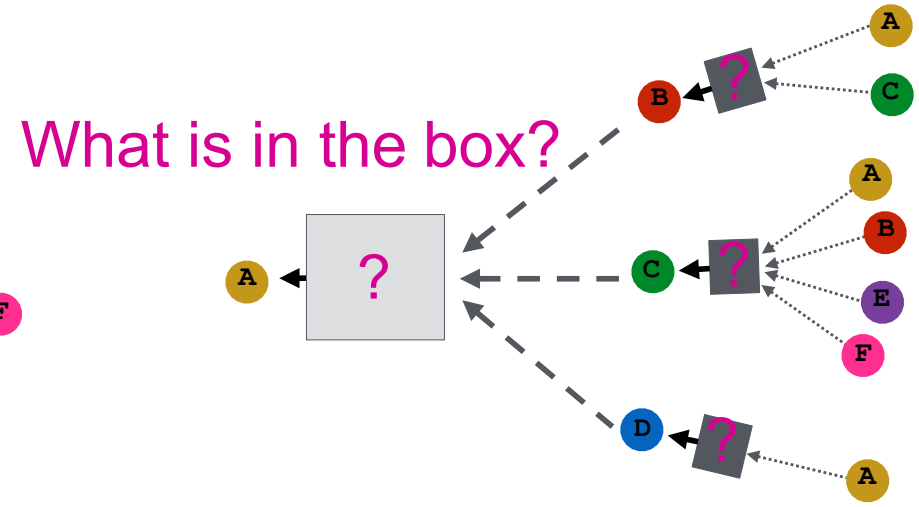
INPUT GRAPH



Neighborhood Aggregation

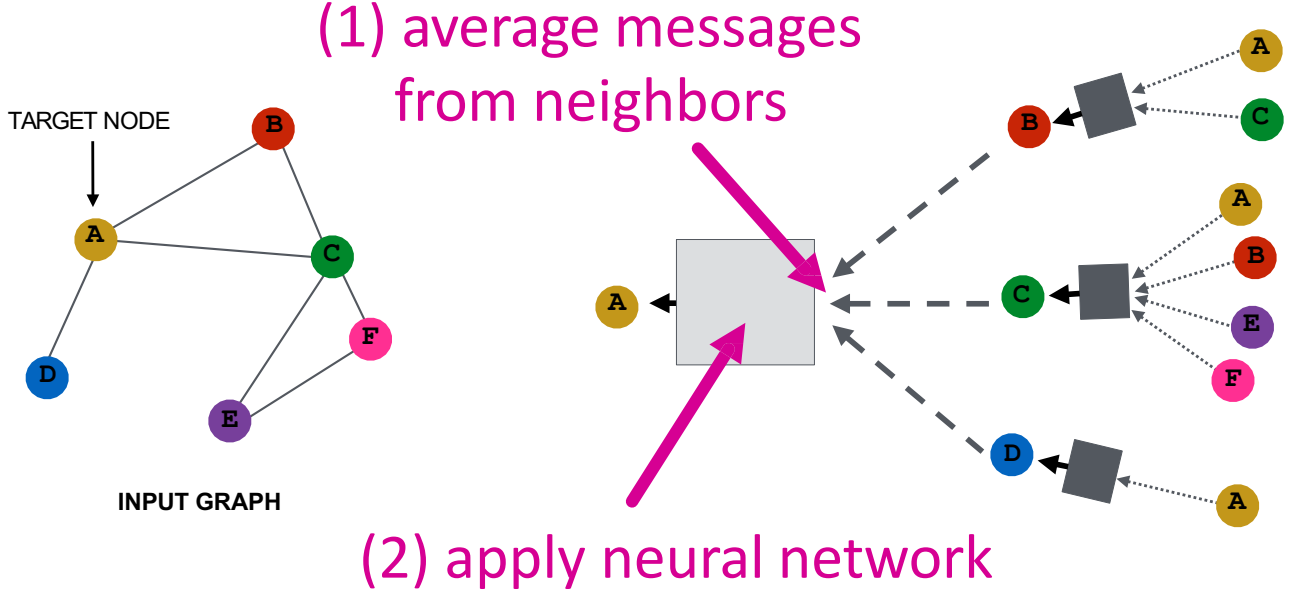
Neighborhood aggregation: Key distinctions are in how different approaches aggregate information across the layers





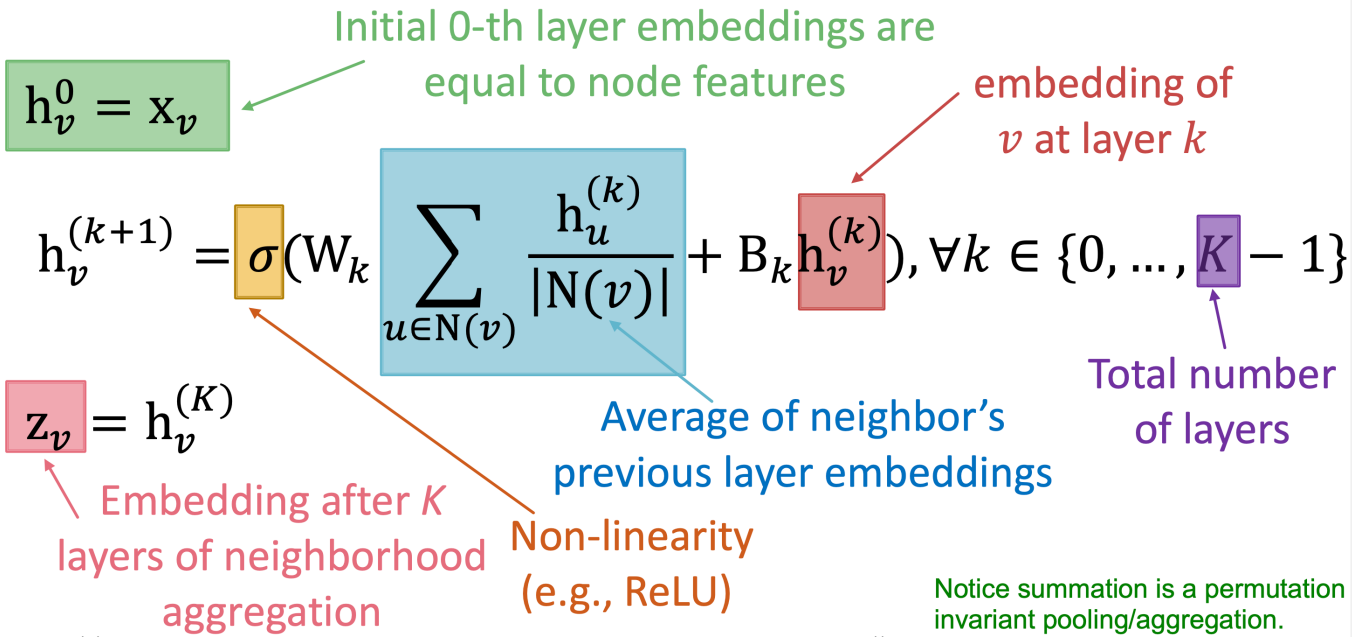
Neighborhood Aggregation

Basic approach: Average information from neighbors and apply a neural network



The Math: Deep Encoder

- **Basic approach:** Average neighbor messages and apply a neural network



The Next Four Pages Show Information Flow

We use a fixed toy update so that every number can be followed.

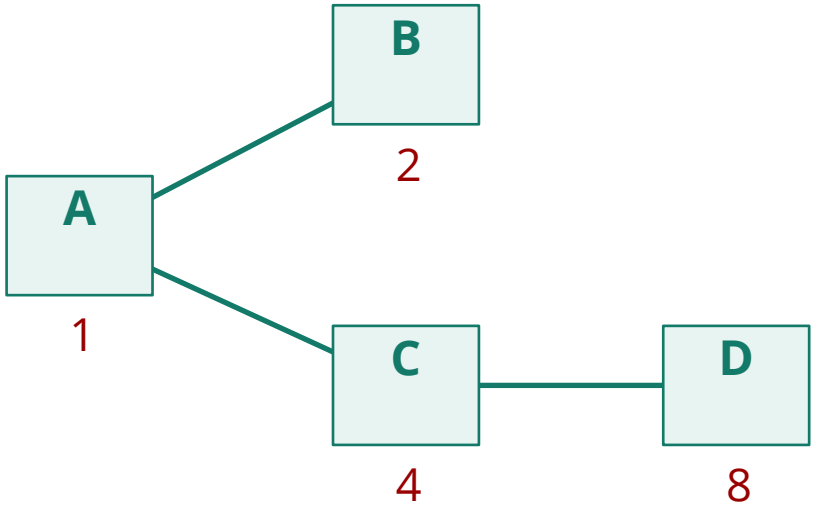
Layer 0	Observed load indices: A = 1, B = 2, C = 4, D = 8.
Toy update	Half of the own value + half of the neighbor mean.
Layer 1 and later	Computed hidden values. They are no longer vehicle counts.

After an update, the number is a hidden value used by the model.

Course schematic. Numbers are illustrative; sources in speaker notes.

Read Only One-Hop Neighbors

All nodes: layer 0



A reads B and C.

Neighbor mean $= (2 + 4)/2 = 3$

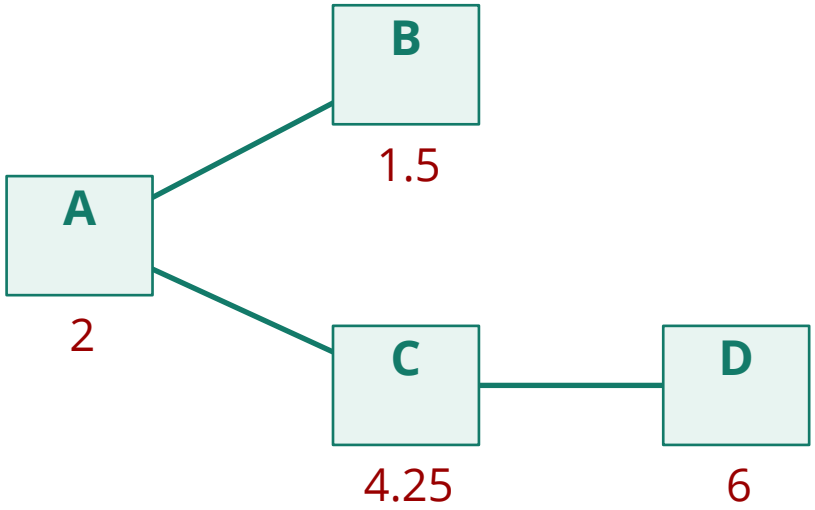
$A^1 = 0.5 \cdot 1 + 0.5 \cdot 3 = 2$

$h_v^{k+1} = 0.5\, h_v^k + 0.5\, \text{mean}\{h_u^k : u \text{ is a neighbor of } v\}$

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Update Every Node From Layer 0

All nodes: layer 1



All layer-1 values:

$A^1 = 2, B^1 = 1.5$

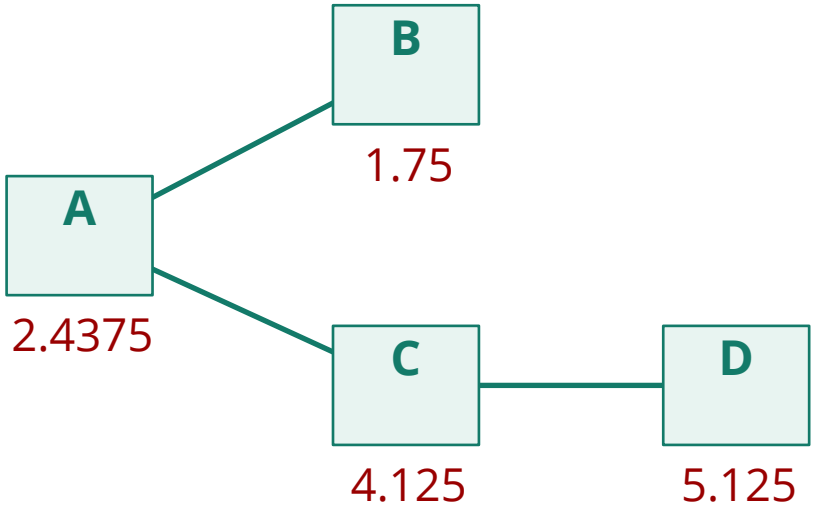
$C^1 = 4.25, D^1 = 6$

Every new value uses only the previous layer.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

The Second Layer Reaches Two Hops

All nodes: layer 2



A reads B^1 and C^1.

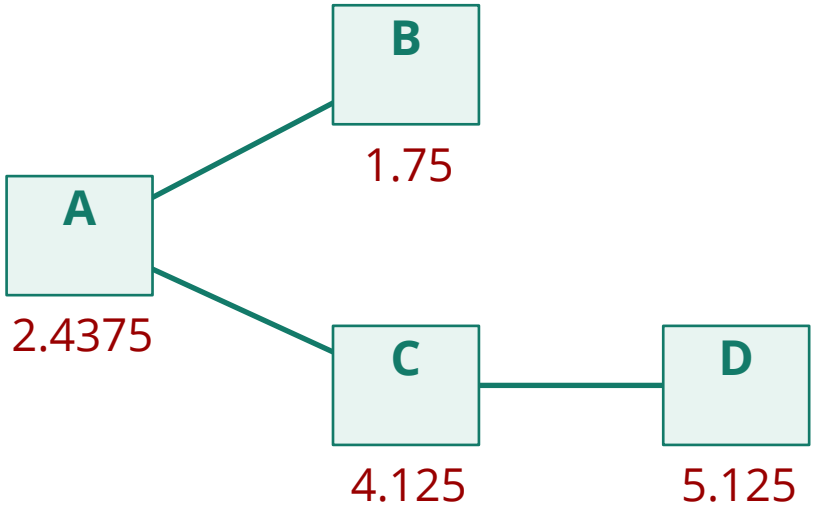
$$A^2 = 0.5 \cdot 2 + 0.5 \cdot (1.5 + 4.25)/2 = 2.4375$$

Two layers let information travel two graph edges.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Keep Updates Synchronous

All nodes: layer 2



Read layer k → write layer k+1

In-place A → B would give
$B^1 = 2$ instead of 1.5.

Node numbering does not define an update schedule.

Toy mean-aggregation update with fixed weights; not the exact normalized GCN rule.

Which Parts of the Update Can Learn?

Our 0.5 / 0.5 example explains information flow with fixed weights.

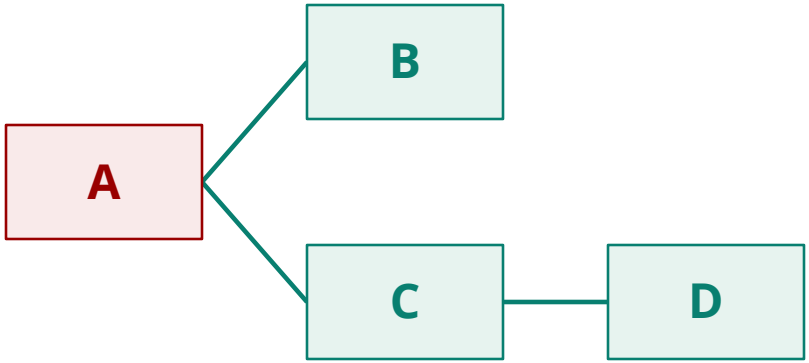
Neighbor information	Transform or weight the messages.
Node information	Combine messages with the node's current vector.
Shared parameters	Reuse the same update rule across nodes.

Training learns shared transforms from the prediction loss.

Course schematic. GNN sources and assumptions in speaker notes.

One Layer and Two Layers Use Different Context

Information moves one graph edge per local aggregation layer.



Every node reads the previous layer.

Layer 1 at A: B and C

Layer 2 at A: D can contribute through C.

Depth expands the neighborhood that can influence a node.

Course schematic. GNN sources and assumptions in speaker notes.

Neighbor Order Should Preserve A's Update

Road A has the same two neighbors whether we list them as B, C or C, B.

Neighbor Listing	Values Read	Mean
B, C	2, 4	3
C, B	4, 2	3

Toy update for A: $0.5 \times 1 + 0.5 \times 3 = 2$ in both cases.

A mean or sum treats the neighbors as a set.

Course schematic. Numbers are illustrative; sources in speaker notes.

Node Outputs Are Equivariant

A GNN gives one hidden vector per road. The output rows follow the input rows.

File Order	Layer-1 Outputs in That Order
A, B, C, D	2, 1.5, 4.25, 6
D, B, A, C	6, 1.5, 2, 4.25

Road A still has output 2 after its row moves.

Course schematic. Numbers are illustrative; sources in speaker notes.

A Whole-Graph Output Can Be Invariant

For a graph task, combine all node outputs into one graph vector.

Sum the layer-1 values as a simple graph representation:

Row Order	Pooled Sum
A, B, C, D	2 + 1.5 + 4.25 + 6 = 13.75
D, B, A, C	6 + 1.5 + 2 + 4.25 = 13.75

The same set of node vectors gives the same pooled result.

Course schematic. Numbers are illustrative; sources in speaker notes.

Invariance and Equivariance Answer Different Questions

Always say what is being reordered and what the output represents.

Operation	Reordering	Required Behavior
Neighbor mean at A	B, C → C, B	Same aggregate
All node outputs	ABCD → DBAC	Rows move together
Whole-graph pooling	ABCD → DBAC	Same graph vector

Reordering the file keeps the same physical prediction.

Course schematic. Numbers are illustrative; sources in speaker notes.

Changing an Edge Changes the Input Graph

Adding an A–D connection is different from changing the file order.

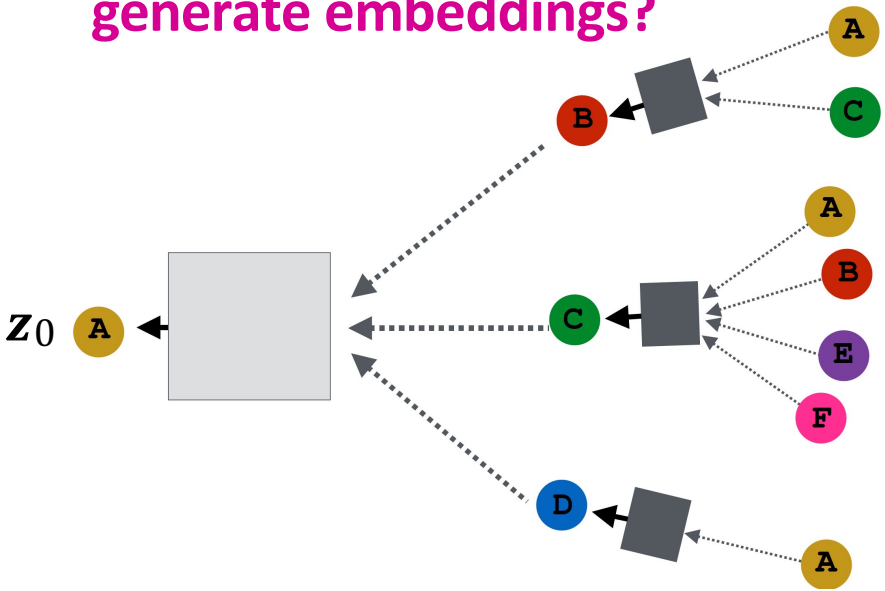
Original graph	A reads B and C. Their mean is 3.
Add connection A–D	A also reads D = 8. The neighbor mean becomes about 4.67.
Possible new output	The toy update for A changes from 2 to about 2.83.

A graph model can respond to changed connections.

Course schematic. Numbers are illustrative; sources in speaker notes.

Training the Model

How do we train the GCN to generate embeddings?



Need to define a loss function on the embeddings.

Model Parameters

Trainable weight matrices
(i.e., what we learn)

$$h_v^{(0)} = x_v$$

$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

$$z_v = h_v^{(K)}$$

Final node embedding

We can feed these **embeddings into any loss function** and run SGD to **train the weight parameters**

h_v^k: the hidden representation of node v at layer k

- W_k: weight matrix for neighborhood aggregation
- B_k: weight matrix for transforming hidden vector of self

The Node Vector Feeds a Prediction Head

For node A, the head converts h_A into class scores.



A training label supplies the loss for a labeled node.

The head and GNN layers train together when both are trainable.

Course schematic. GNN sources and assumptions in speaker notes.

How to Train A GNN

Node embedding $\boldsymbol{z}_0$ is a function of input graph

Supervised setting: We want to minimize loss $\mathcal{L}$:

$$\min_{\Theta} \mathcal{L}(\boldsymbol{y}, f_{\Theta}(\boldsymbol{z}_v))$$

- $\boldsymbol{y}$: node label
- $\mathcal{L}$ could be L2 if $\boldsymbol{y}$ is real number, or cross entropy if $\boldsymbol{y}$ is categorical

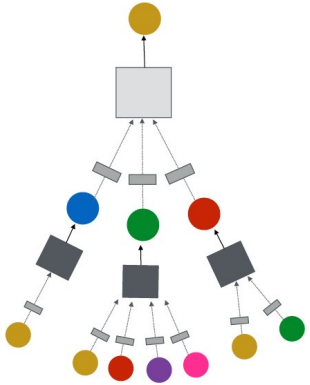
Unsupervised setting:

- No node label available
- **Use the graph structure as the supervision!**

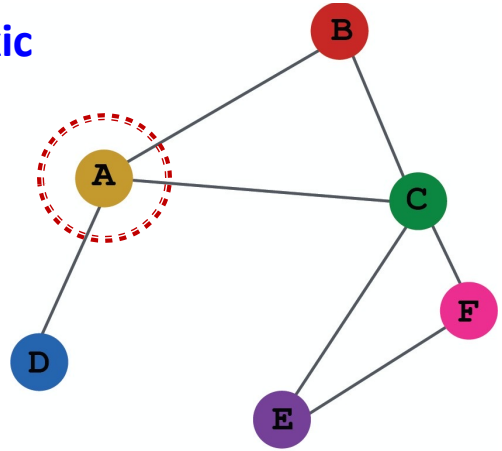
Supervised Training

Directly train the model for a supervised task (e.g., **node classification**)

Safe or toxic drug?



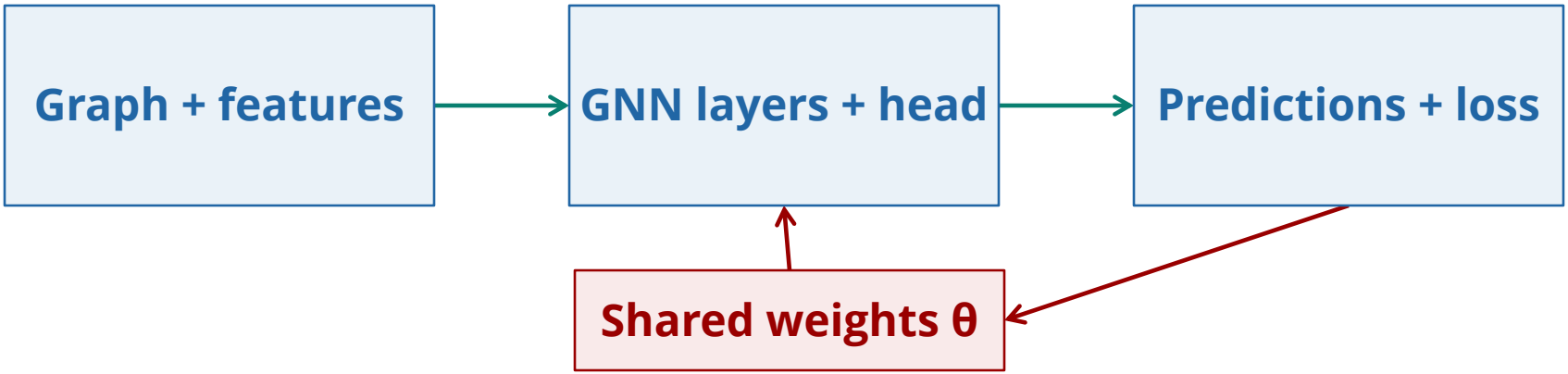
Safe or toxic drug?



E.g., a drug-drug interaction network

A Complete GNN Training Step

Connections define the computation; labels tell us how to score it.



Backpropagation updates the shared model parameters.

Course schematic. GNN sources and assumptions in speaker notes.

Supervised Training

Directly train the model for a supervised task (e.g., **node classification**)

- Use cross entropy loss

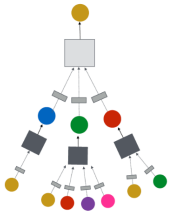
$$\mathcal{L} = -\sum_{v \in V} y_v \log(\sigma(z_v^T \theta)) + (1 - y_v)\log(1 - \sigma(z_v^T \theta))$$

Encoder output: node embedding

Classification weights

Node class label

Safe or toxic drug?



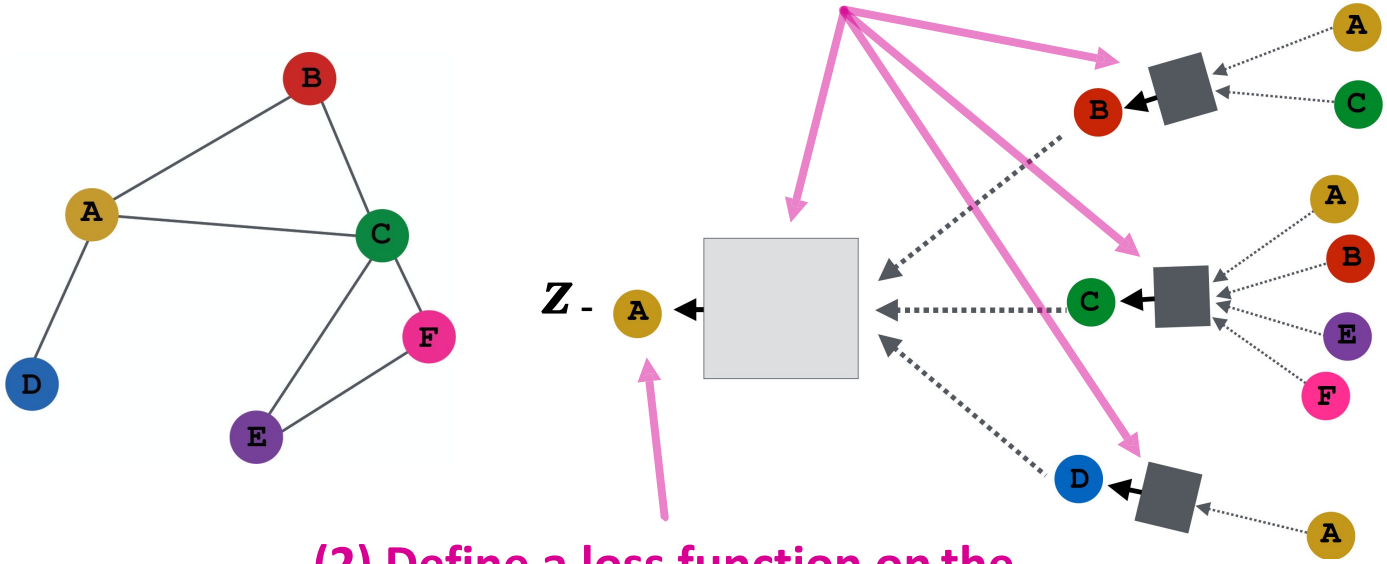
True probability distribution (one-shot)

$$H(p, q) = -\sum_{x \in classes} p(x) \log q(x)$$

Your model's predicted probability distribution

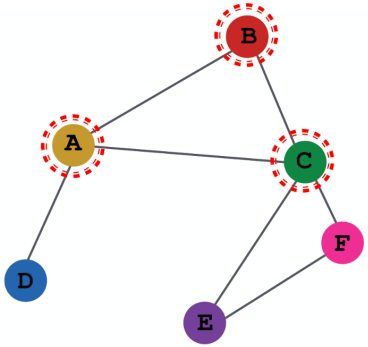
Model Design: Overview

(1) Define a neighborhood aggregation function



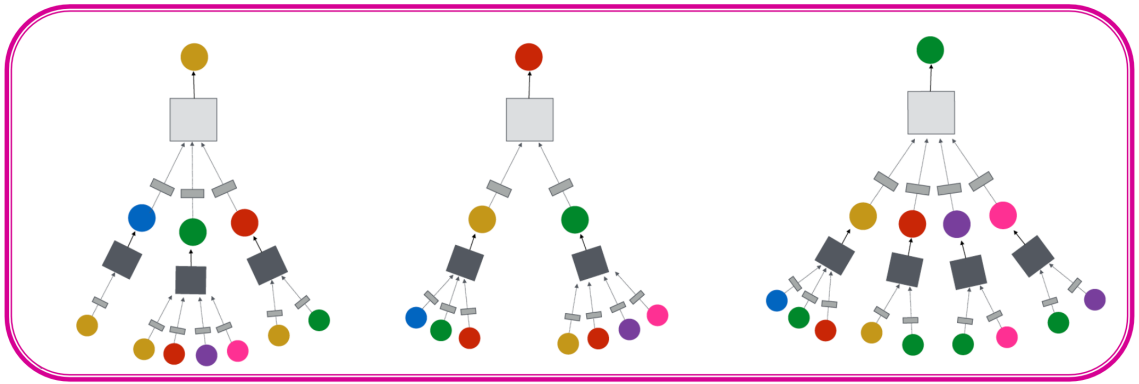
(2) Define a loss function on the embeddings

Model Design: Overview

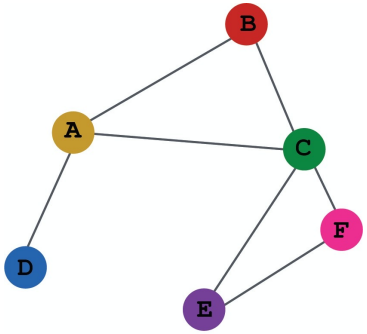


INPUT GRAPH

(3) Train on a set of nodes, i.e., a batch of compute graphs



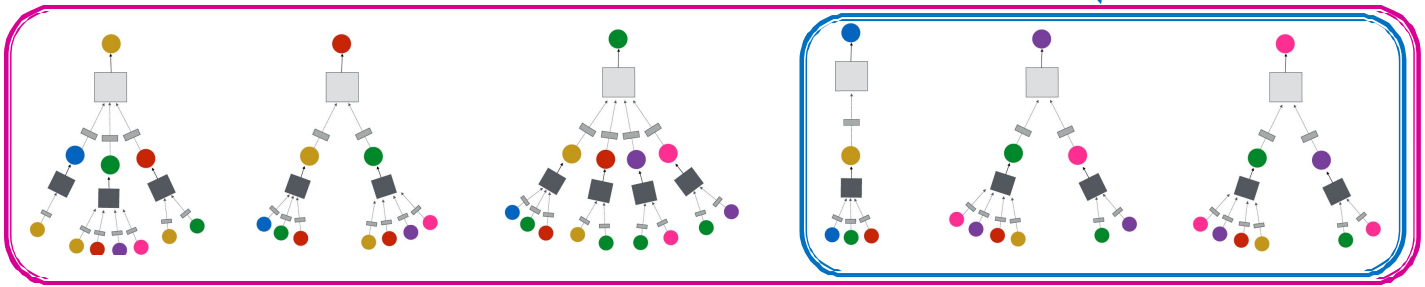
Model Design: Overview



INPUT GRAPH

(4) Generate embeddings for nodes as needed

Even for nodes we never trained on!



What Is Shared Across Roads?

At one GNN layer, every road uses the same learned matrices.

Road-specific inputs	A reads its own vector and its neighbor mean. B reads its own.
Shared learned weights	W_self transforms a node vector. W_neighbor transforms a mean.
Repeated computation	Each road runs the same update with different input vectors.

The weights depend on feature width, rather than road IDs.

Course schematic. Numbers are illustrative; sources in speaker notes.

Why Can the Weights Run on a New Graph?

Suppose each node vector has 2 numbers. The learned matrices accept that width.

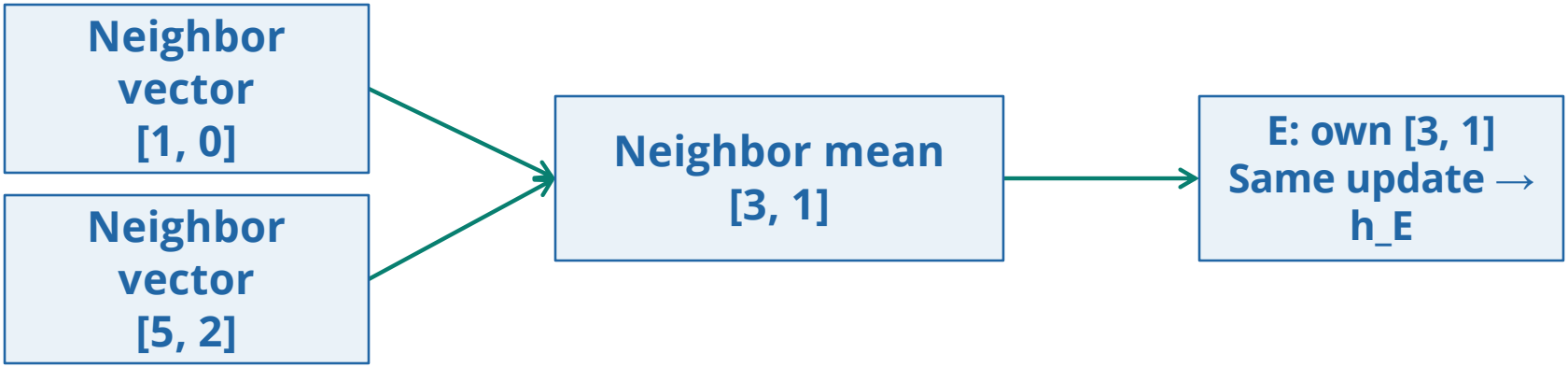
Input to One Update	Width	Shared Transform
Own road vector	2 numbers	W_self: 2 × 2
Mean of neighbor vectors	2 numbers	W_neighbor: 2 × 2
Updated road vector	2 numbers	Same rule at every road

More nodes mean more uses of the same function.

Course schematic. Numbers are illustrative; sources in speaker notes.

A New Road E Can Be Processed

E and its neighbors supply vectors with the same width the encoder expects.



The learned matrices can process E because the vector widths match.

Course schematic. Numbers are illustrative; sources in speaker notes.

Inductive Means Applying the Learned Rule to Unseen Nodes

Training fits the encoder. Inference computes outputs for new compatible inputs.

Training city	Use road inputs, links and observed delay labels to fit the weights.
New city	Keep the weights. Compute vectors from its road inputs and links.
Check generalization	Test accuracy, feature units, edge meanings and traffic conditions.

Computing an output and predicting accurately are separate checks.

Course schematic. Numbers are illustrative; sources in speaker notes.

This Is the Same Reuse Idea as a CNN

A trained image filter can scan a new image. A trained GNN update can visit new nodes.

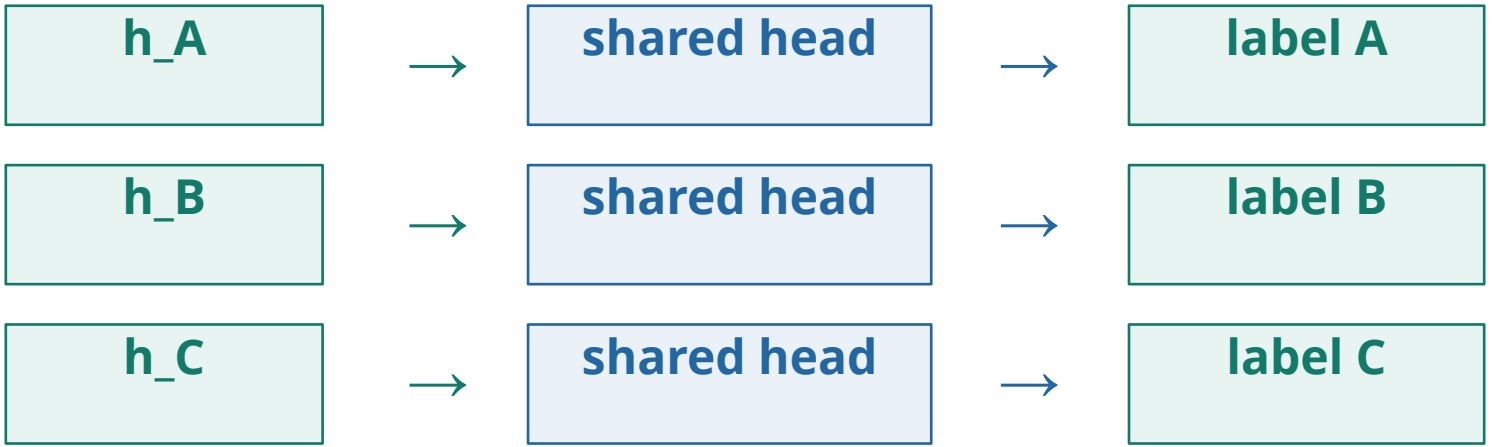
CNN encoder	Reuse learned filters at many pixels and on new images.
GNN encoder	Reuse learned update weights at many nodes and on new graphs.
Common requirement	The new inputs must have the expected feature meanings and widths.

The learned operation is reusable across compatible inputs.

Course schematic. Numbers are illustrative; sources in speaker notes.

Node Outputs Keep Their Identity

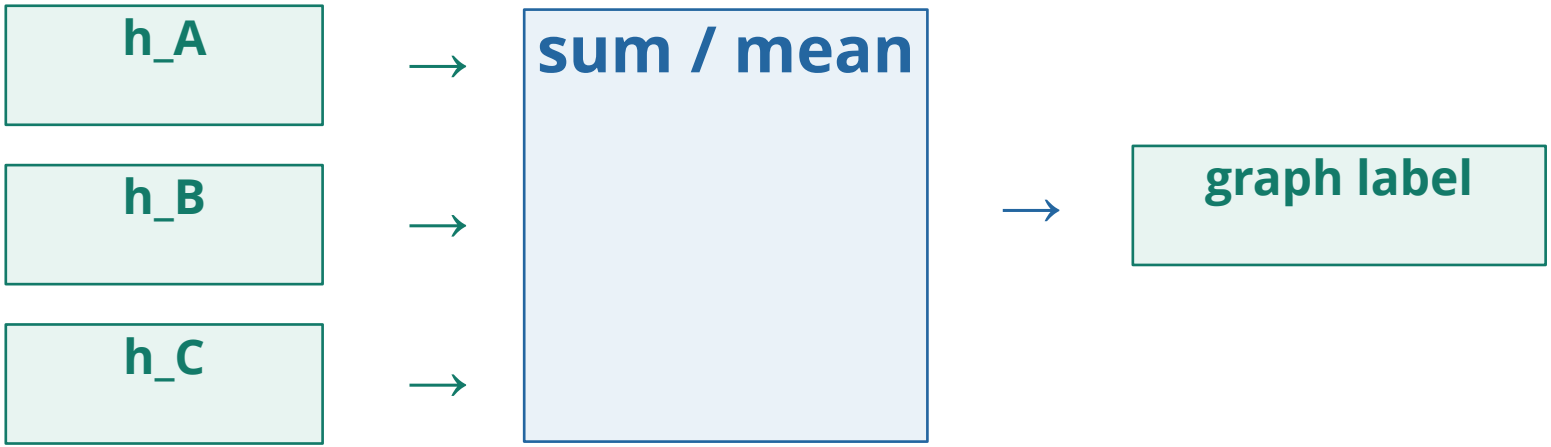
1 / 2



Reordering node inputs reorders the corresponding node outputs.

Course schematic.

Graph Outputs Pool Across Nodes

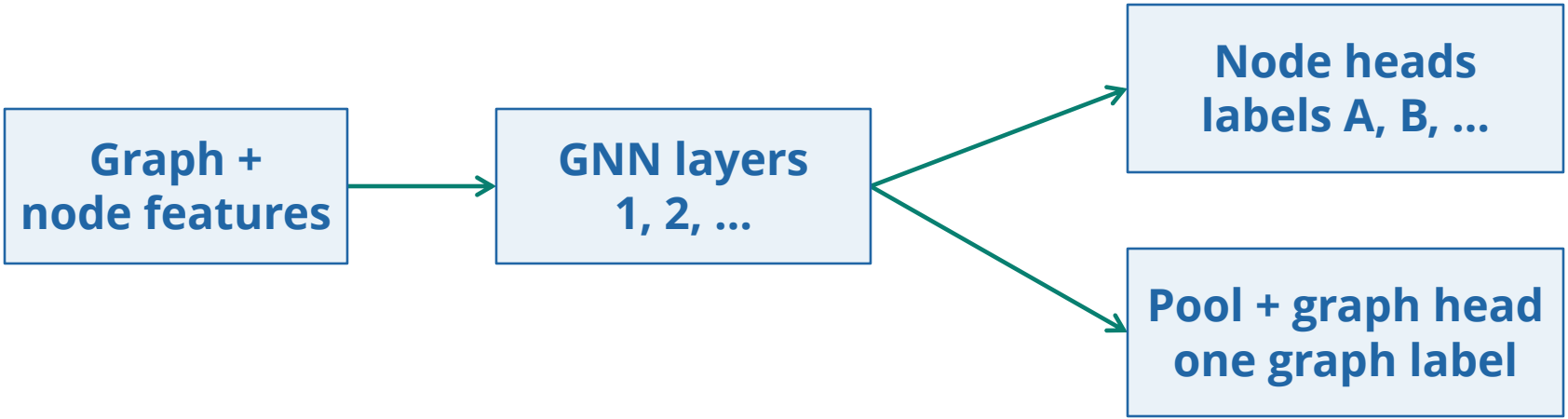


A symmetric readout gives the same graph output after reordering.

Course schematic.

The GNN From Input to Output

Node tasks keep one output per node. Graph tasks pool the node vectors.

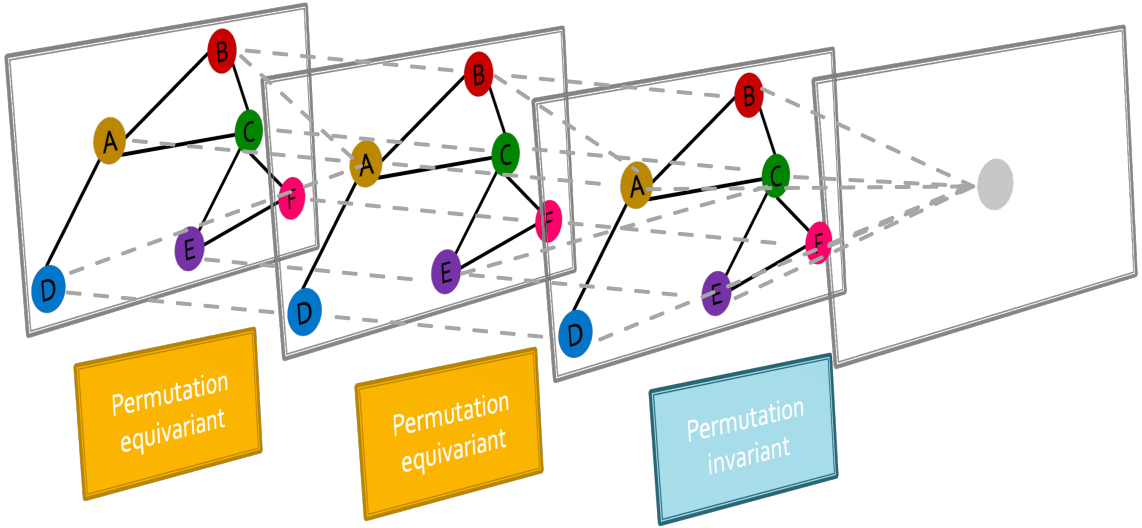


The output task determines whether we keep or pool node vectors.

Course schematic. GNN sources and assumptions in speaker notes.

GNN Layers and the Graph Readout

Two node-update layers feed a graph readout that combines the node vectors.



Equivariant node layers. Invariant graph pooling.

Course example. Sources and assumptions in speaker notes.

More Layers Are a Modeling Choice

A larger neighborhood can help, but it can also dilute differences.

One or two layers	Use nearby graph context.
More layers	Reach more distant nodes and reuse transformed signals.
Possible cost	Node vectors may become less distinct; compute increases.

Choose depth using the task and validation results.

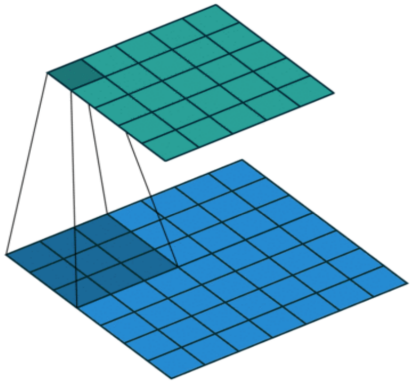
Course schematic. GNN sources and assumptions in speaker notes.

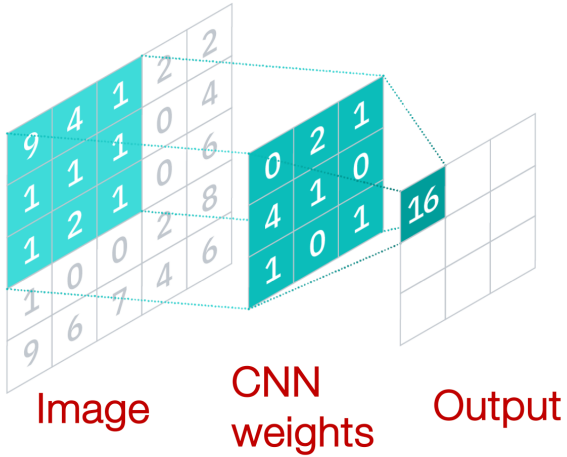
Architecture Comparison

How do GNNs compare to prominent architectures such as Convolutional Neural Nets?

Convolutional Neural Network

Convolutional neural network (CNN) layer with 3x3 filter:



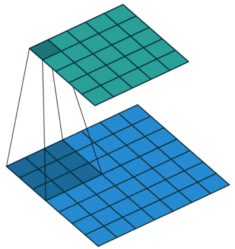


CNN formulation: $\mathrm{h}_v^{(l+1)} = \sigma(\sum_{u \in \mathrm{N}(v) \cup \{v\}} \mathrm{W}_l^u \mathrm{h}_u^{(l)}), \quad \forall l \in \{0, \ldots, L-1\}$

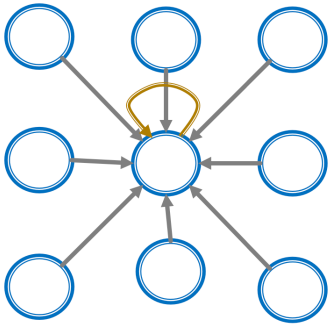
$N(v)$ represents the 8 neighbor pixels of v.

GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3 filter:



Image



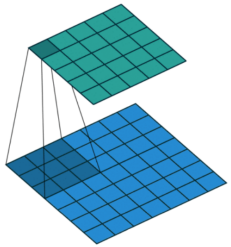
Graph

- GNN formulation: $\mathrm{h}_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in \mathrm{N}(v)} \frac{\mathrm{h}_u^{(l)}}{|\mathrm{N}(v)|} + \mathrm{B}_l \mathrm{h}_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$
- CNN formulation: (previous slide) $\mathrm{h}_v^{(l+1)} = \sigma(\sum_{u \in \mathrm{N}(v) \cup \{v\}} \mathrm{W}_l^u \mathrm{h}_u^{(l)}), \forall l \in \{0, \ldots, L-1\}$

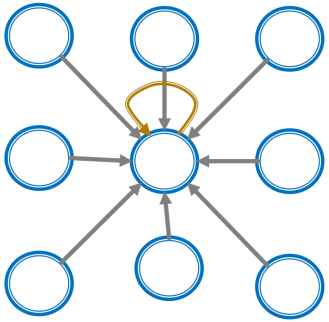
  if we rewrite: $\mathrm{h}_v^{(l+1)} = \sigma(\sum_{u \in \mathrm{N}(v)} \mathbf{W}_l^u \mathrm{h}_u^{(l)} + \mathrm{B}_l \mathrm{h}_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$

GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3 filter:



Image



Graph

GNN formulation: $\mathrm{h}_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in \mathrm{N}(v)} \frac{\mathrm{h}_u^{(l)}}{|\mathrm{N}(v)|} + \mathrm{B}_l \mathrm{h}_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$

CNN formulation: $\mathrm{h}_v^{(l+1)} = \sigma(\sum_{u \in \mathrm{N}(v)} \mathbf{W}_l^u \mathrm{h}_u^{(l)} + \mathrm{B}_l \mathrm{h}_v^{(l)}), \forall l \in \{0, \ldots, L-1\}$

Key difference: We can learn different W_l^u for different "neighbor" u for pixel v on the image. The reason is we can pick an order for the 9 neighbors using **relative position** to the center pixel: {(-1,-1). (-1,0), (-1, 1), …, (1, 1)}

Attention Connects Allowed Token Positions

In Block 5, tokens gathered context from allowed positions.

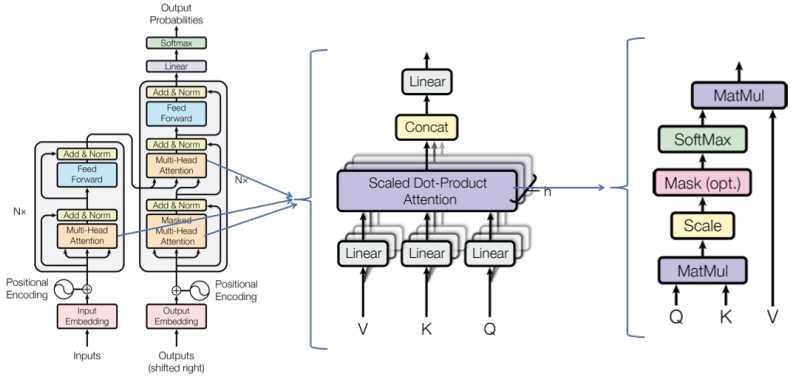
Road GNN	Read the connected road segments.
Self-attention	Read the token positions allowed by the mask.
Different weighting	Attention learns input-dependent mixing weights.

GNN neighborhoods and attention masks both restrict information flow.

Course schematic. GNN sources and assumptions in speaker notes.

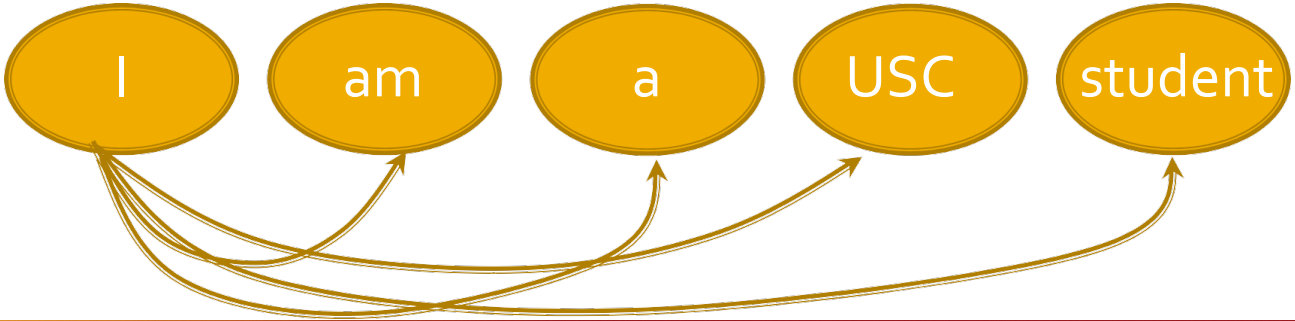
Transformer

Transformer is one of the most popular architectures that achieves great performance in many sequence modeling tasks.



Key component: self-attention

Each token attends to allowed positions. A causal mask excludes future tokens.



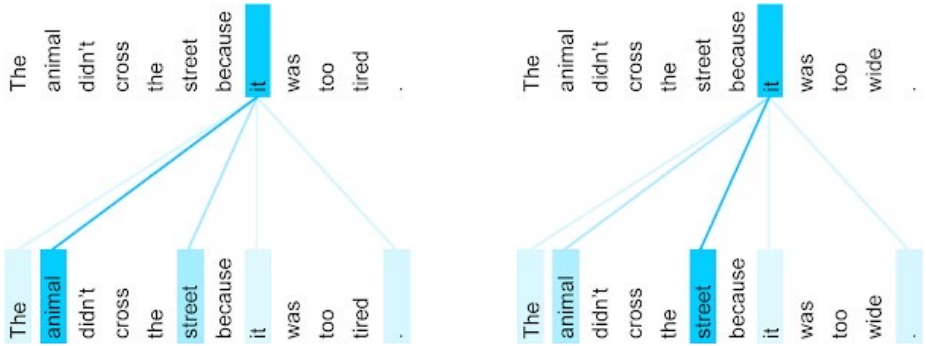
USC Viterbi School of Engineering

Transformer

A general definition of attention:

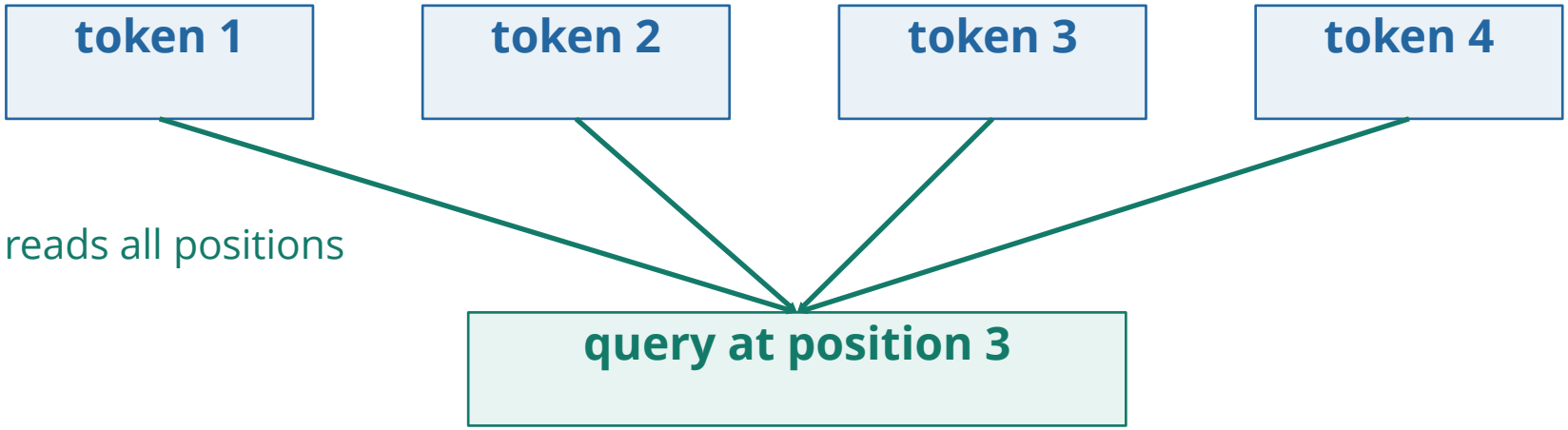
Given a set of vector *values*, and a vector *query*, **attention** is a technique to compute a weighted sum of the values, dependent on the query.

Each token has a query, key and value. Compare a query with allowed keys to obtain weights. Use those weights to mix the corresponding values.



Attention Defines Allowed Edges

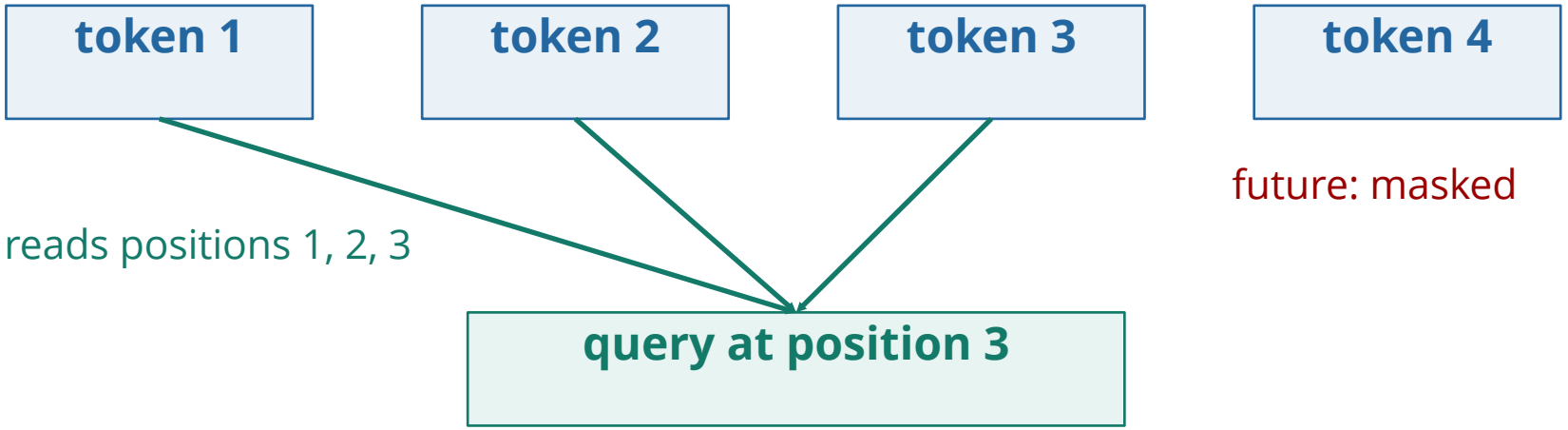
1 / 2



Unmasked self-attention uses a complete token interaction graph.

Course schematic.

Causal Attention Removes Future Edges



The causal mask makes the allowed interactions directional.

Course schematic.

Graphs and Language Models: Two Tasks

We know how a GNN reads neighbors and how a language model reads text.

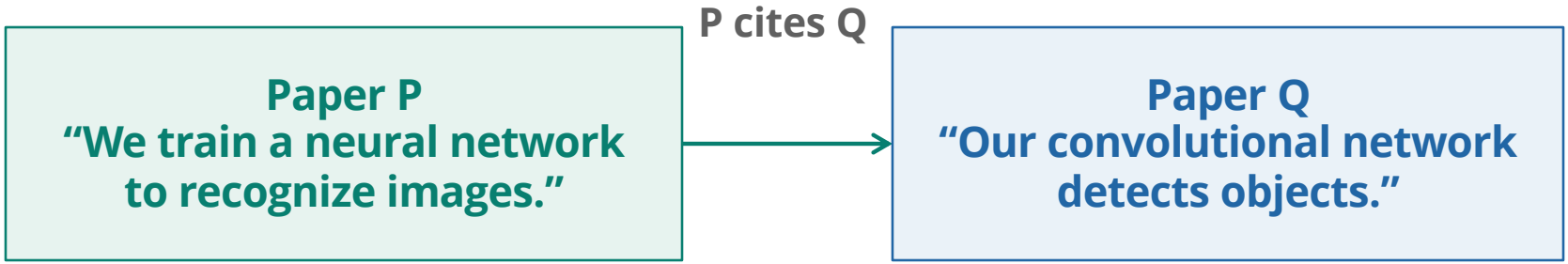
Paper classification	Input: abstracts + citation links. Output: a topic label.
Document question answering	Input: a question + documents. Output: an answer with sources.
Agent examples afterward	Use graphs to describe actions and communication paths.

First classify papers. Then retrieve evidence to answer a question.

Teaching example based on Chen et al. (2023/2024).

A Paper Has Text and Citation Neighbors

The task is to classify paper P as AI or Biology in this toy example.



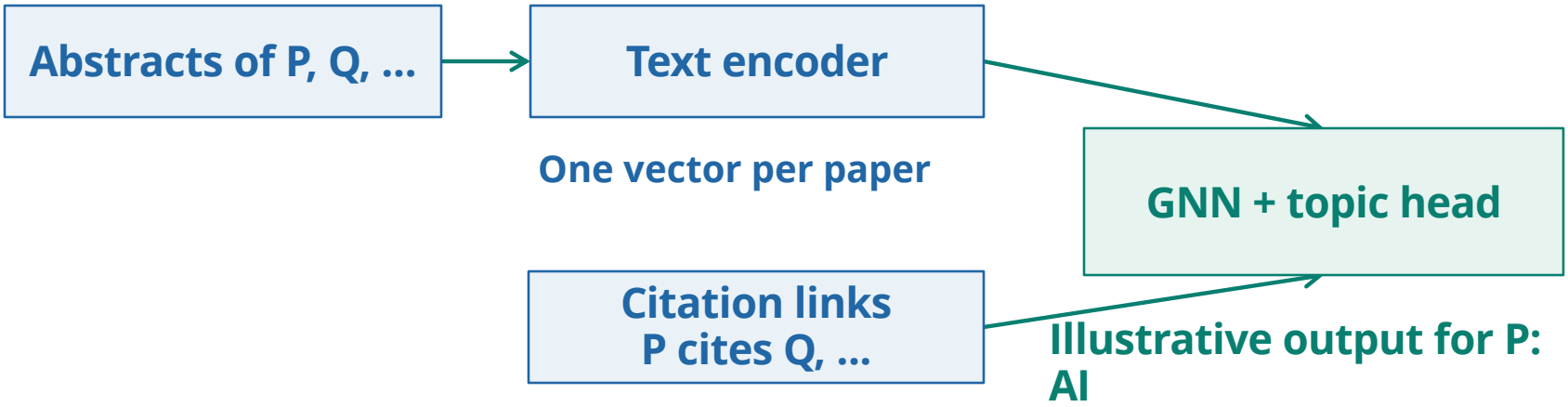
Predict the topic of P using its abstract and available citation context.

Use both the paper’s text and its citation neighbors.

Teaching example based on Chen et al. (2023/2024).

Route 1: Text Features Feed a GNN

A text encoder turns each abstract into a vector. Citation links define neighbors.



The GNN prediction head produces the topic label.

Teaching example based on Chen et al. (2023/2024).

Follow One Paper Through the Enhancer Pipeline

Paper P says: “We train a neural network to recognize images.”

Encode P’s text	A numeric vector, illustrated as [0.2, 0.8].
Read citations	The GNN also gathers vectors from P’s graph neighbors.
Predict P’s topic	The prediction head uses the updated vector to output a label.

The final topic label comes from the GNN prediction head.

Teaching example based on Chen et al. (2023/2024).

What Gets Trained in the Enhancer Pipeline?

One simple setup keeps the text encoder fixed while training the graph predictor.

Text encoder	Produces one input vector per paper. Keep its weights fixed here.
GNN and head	Learn from labeled training papers using the prediction loss.
New paper	Encode its text, gather neighbors, and run the trained graph predictor.

The training labels teach the GNN how to use those features.

Teaching example based on Chen et al. (2023/2024).

Pipeline 2: Ask the LLM for the Topic

Give the model a classification question and the paper's text.

Prompt to the LLM

Choose AI or Biology.
Paper P: "We train a neural network to recognize images."
Topic:

Illustrative answer: AI

Here, the LLM produces the final topic label.

Teaching example based on Chen et al. (2023/2024).

How Can Graph Context Enter That Prompt?

A citation edge is converted into a readable sentence about a neighbor.

Paper P cites Q.
Q's abstract: "Our convolutional network detects objects."
Use P's abstract and this cited-paper summary to classify P.

Compare text-only and text-plus-neighbor inputs on the same held-out papers.

The LLM can only read the graph information we include.

Teaching example based on Chen et al. (2023/2024).

Two Routes to the Same Topic Label

Both routes answer the same question about paper P. Follow the final predictor.

Route 1	Abstracts → text vectors → GNN + head → topic label.
Route 2	Abstract + cited-paper summary → LLM → topic label.
What differs?	Who reads the graph context and who produces the final label.

We can compare the routes on the same held-out papers.

Teaching example based on Chen et al. (2023/2024).

Check Which Part of a Pipeline Produced the Result

For the paper-topic task, we can change one input while keeping the test fixed.

Text-only baseline	How well does the paper's own abstract predict its topic?
Add graph context	Does adding cited-paper information improve the same test?
Check leakage	Were test labels accidentally included in any features or prompt?

Use matched comparisons to test the value of graph context.

Teaching example based on Chen et al. (2023/2024).

A New Task: Answering From Documents

Question: Which apps could be affected if the Atlas database goes offline?

What we just did	Predict one paper's topic from text and citation context.
What changes now	Answer a user's question using evidence in several documents.
Why a graph may help	One document names a service. Other documents name its apps.

We now need supporting facts spread across documents.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

RAG Finds Documents Before the LLM Answers

RAG means retrieval-augmented generation. Retrieval supplies relevant evidence.



Our question: Which apps could be affected if the Atlas database goes offline?

The LLM answers using the documents that were retrieved.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

The Needed Facts Are in Different Documents

Atlas and Cedar are databases; Beacon is a service; HelpDesk, Alerts and Search are apps.

Document	Recorded Fact
D1	Beacon uses data from the Atlas database.
D2	The HelpDesk app uses data from Beacon.
D3	The Alerts app uses data from Beacon.
D4	The Search app uses data from the Cedar database.

D1 connects Atlas to Beacon. D2 and D3 connect Beacon to apps.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

What Happens If Retrieval Finds Only D1?

Suppose the initial search returns the document mentioning “Atlas.”

Retrieved D1	“Beacon uses data from the Atlas database.”
What D1 tells us	Beacon depends on Atlas.
What is still missing	Which apps depend on Beacon? We need more documents.

D1 identifies Beacon, but the app names are in D2 and D3.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Index Documents, Then Answer Questions

The index is reusable. Each question selects the evidence it needs.

Before questions	Read documents. Store systems, relations and source IDs.
For this question	Locate Atlas. Find connected facts and fetch their source passages.
Answer	Give the question and selected passages to the LLM.

Build the graph first. Then retrieve evidence for each question.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

One Document Becomes One Sourced Relation

D1 says: “Beacon uses data from the Atlas database.”



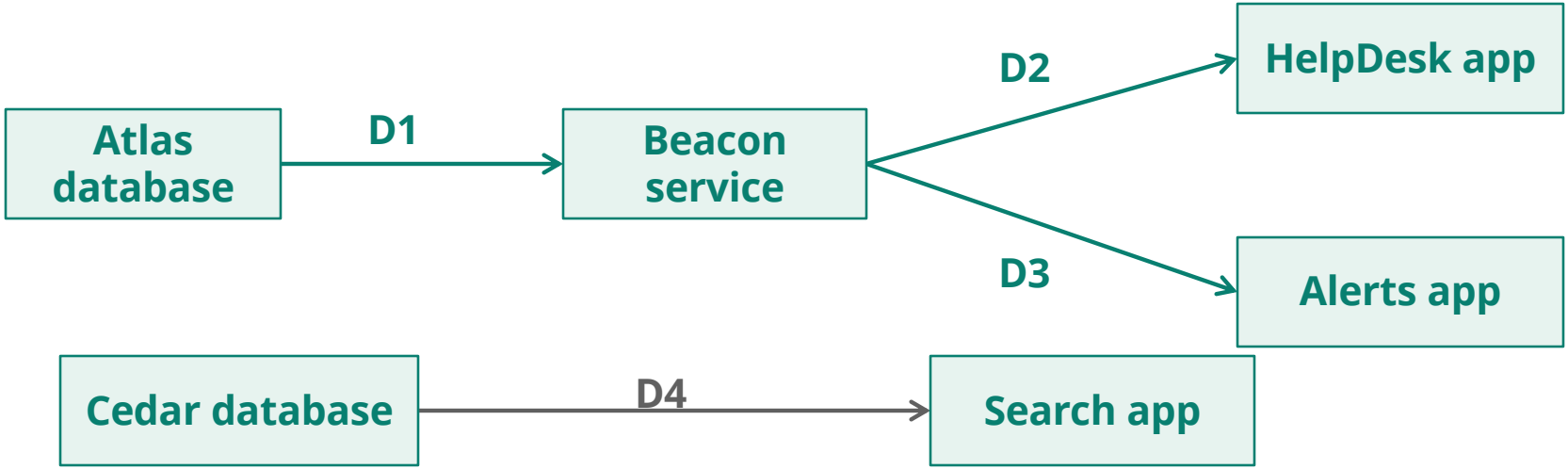
Store: (Atlas, supplies data to, Beacon, D1)

Here an arrow points from a data supplier to its consumer.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

The Documents Form Two Connected Components

Each edge records a document fact. D1, D2 and D3 connect Atlas to two apps.



D4 links Search to Cedar in a separate part of the graph.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

An Edge Is a Claim With a Source

The system stores both the relation and a link back to the supporting document.

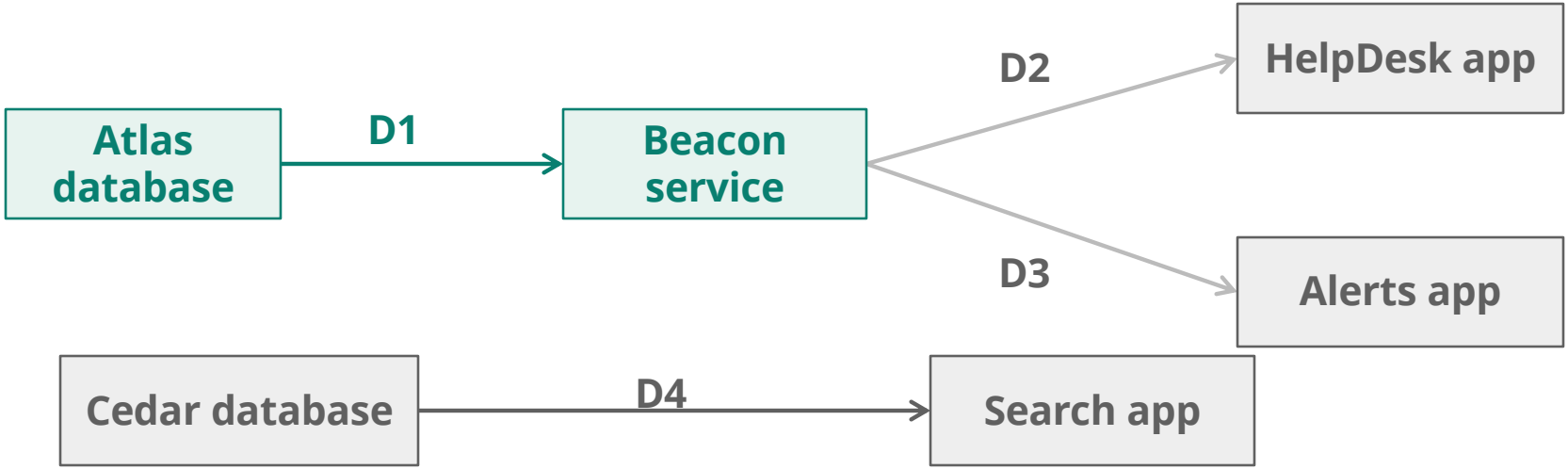
Supplier	Uses Its Data	Source
Atlas	Beacon	D1
Beacon	HelpDesk	D2
Beacon	Alerts	D3
Cedar	Search	D4

The graph helps us find facts; the source text lets us check them.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Hop 1: Atlas Supplies Beacon

Locate the database named in the question, then follow supplier-to-consumer edges.

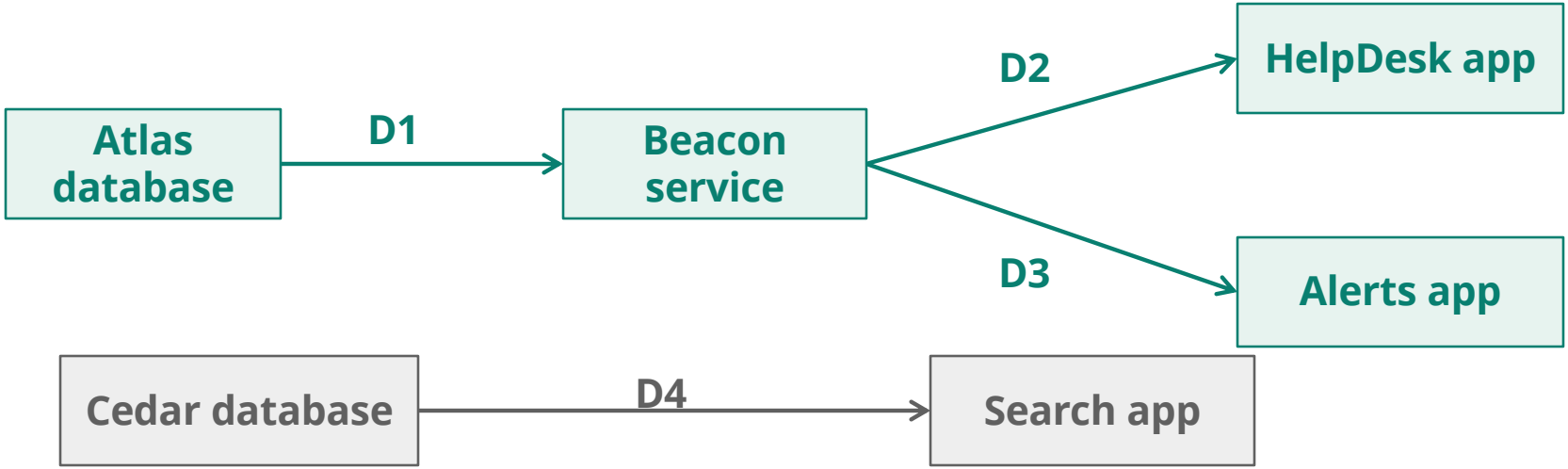


The first hop finds Beacon and the supporting document D1.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Hop 2: Beacon Supplies HelpDesk and Alerts

Continue from Beacon. The next edges reveal the two downstream apps.



The relevant evidence is D1, D2 and D3.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Retrieve the Original Text for Those Edges

Finding a path tells the system which source documents to bring back.

Found Relation	Document Returned
Atlas → Beacon	D1: Beacon uses data from Atlas.
Beacon → HelpDesk	D2: HelpDesk uses data from Beacon.
Beacon → Alerts	D3: Alerts uses data from Beacon.

The selected context is the text of D1, D2 and D3.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

What Exactly Does the LLM Receive?

The system puts the question and the retrieved passages into a prompt.

Question: Which apps could be affected if Atlas goes offline?

Evidence:
[D1] Beacon uses data from the Atlas database.
[D2] The HelpDesk app uses data from Beacon.
[D3] The Alerts app uses data from Beacon.

Use these facts to answer. Cite the document IDs.

The LLM reads evidence text and writes the answer.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Follow the Evidence Into the Answer

Both app names come from retrieved documents. Each path explains why it belongs.

HelpDesk	Could be affected: Atlas → Beacon → HelpDesk. [D1, D2]
Alerts	Could be affected: Atlas → Beacon → Alerts. [D1, D3]
Search	D4 links Search to Cedar; these documents give no Atlas path.

The answer names the apps and cites the dependency chain.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

The Graph and the LLM Do Different Jobs

The graph connects the source facts. The LLM expresses the supported answer.

Graph retrieval	Find the Atlas → Beacon → app paths and their source IDs.
Context assembly	Fetch the matching document passages for the prompt.
LLM generation	Write an answer using the question and those passages.

This example uses graph traversal without training a GNN.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

A Missing Relation Can Remove Needed Evidence

Suppose graph construction misses the D3 relation: Beacon → Alerts.

Stage	What Goes Wrong
Graph construction	D3 exists, but its Beacon → Alerts edge is absent.
Retrieval	The path search returns D1 and D2, missing D3.
Answer	The supported answer may omit Alerts.

Check whether the retrieved evidence covers all required facts.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

A Wrong Relation Can Add an Unsupported App

Suppose the index incorrectly links Atlas to the Search app.

Incorrect graph edge	Atlas → Search appears in the index.
Check the source	D4 says Search uses Cedar. It gives no Atlas relation.
Required correction	Remove or flag the unsupported edge before relying on it.

Verify a graph relation against its source document.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

How Would We Test This Question-Answering System?

For the Atlas example, the required apps are HelpDesk and Alerts.

Required evidence	D1, D2 and D3 establish the two relevant paths.
Required answer	Name both HelpDesk and Alerts, with the supporting source IDs.
Unsupported extra app	Search is absent from the Atlas paths in these documents.

Test evidence retrieval and answer quality separately.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Was the Missing App a Retrieval or Generation Error?

The same incomplete answer can come from different stages.

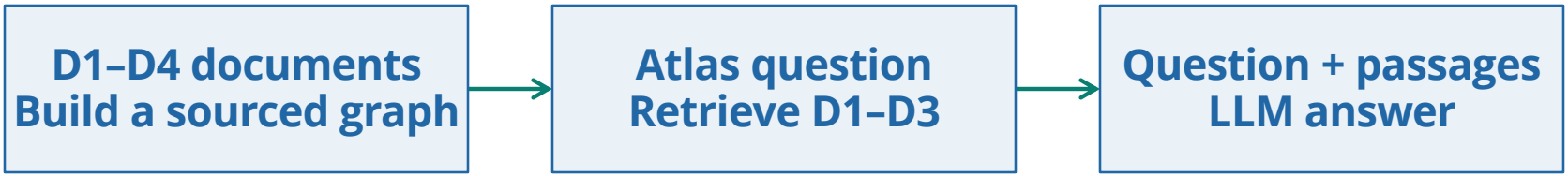
Retrieved Context	Answer	Where to Investigate
D1, D2 only	HelpDesk only	Graph construction / retrieval
D1, D2, D3	HelpDesk only	LLM answer generation

Inspect the retrieved passages before blaming the LLM.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

GraphRAG in Our Atlas Example

The graph selects evidence. The LLM reads that evidence and writes the answer.



Answer: HelpDesk and Alerts could be affected.
Paths: Atlas → Beacon → HelpDesk / Alerts. [D1–D3]

This example retrieves facts without training a GNN.

Fictional systems. Simplified local graph retrieval; Edge et al. (2024).

Graphs Can Also Describe the Agent's Work

The next examples use actions and agents as nodes, rather than document entities.

GraphRAG	An entity is a node. An edge records a document fact for retrieval.
GRADE	An action is a node. An edge records order or data use in a run.
MAMA	An agent is a node. An edge permits messages within a team.

Always identify what an edge means before interpreting a graph.

Course example. Sources and assumptions in speaker notes.

GRADE Makes an Agent Run Easier to Inspect

A run is one sequence of actions performed while completing a task.

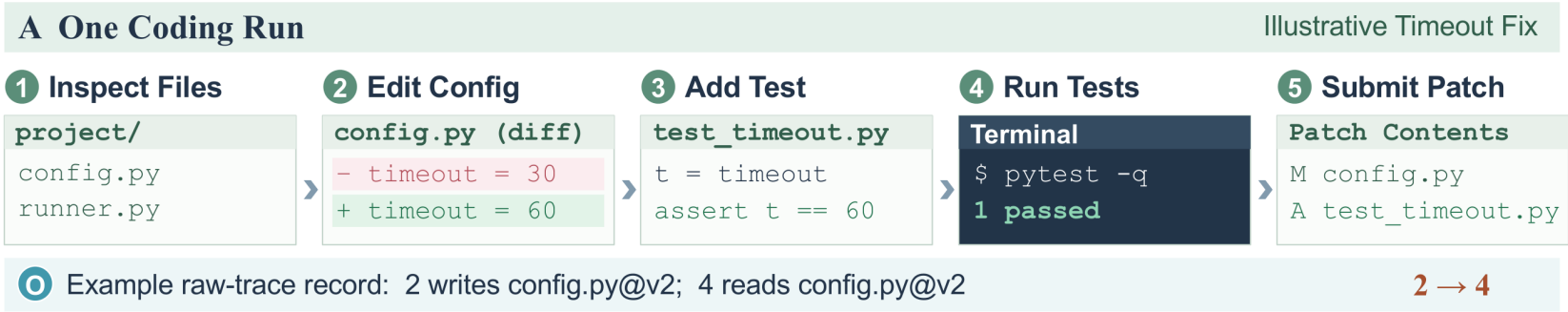
Agent task	Fix a program whose timeout setting is too short.
Run trace	A log of file inspection, edits, tests and patch submission.
Two graph questions	Which action came next? Which action supplied an input?

Record what happened next and which earlier output was used.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

The Coding Example From the GRADE Slides

Five actions: inspect files, edit config, add a test, run tests, submit a patch.



Step 2 writes a new config version; step 4 reads that version.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Read the Five Actions in Plain Language

The agent changes timeout = 30 to timeout = 60, then checks the program.

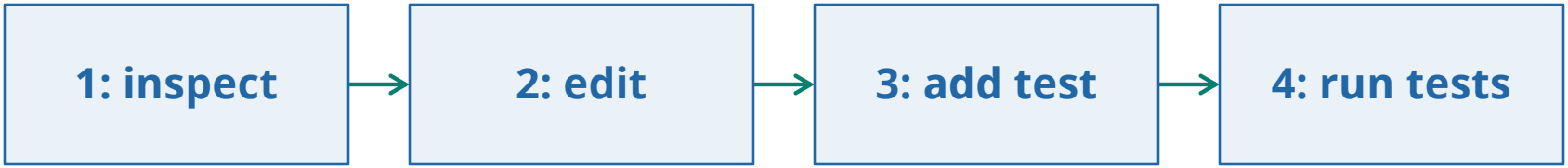
Step	Action	Output or Effect
1	Inspect files	Read the existing code.
2	Edit config	Write config.py with timeout 60.
3	Add a test	Write a test expecting timeout 60.
4, 5	Run tests; submit	Observe test result; submit the patch.

The trace records concrete actions and the files they use.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Execution Edges Answer “What Happened Next?”

An execution edge joins consecutive actions in the logged run.



Then 4 → 5: submit the patch.

The 3 → 4 edge tells us that running tests happened after adding the test.

The execution layer records 1 → 2 → 3 → 4 → 5.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Dependency Edges Answer “Which Output Was Used?”

The trace says step 2 writes config.py@v2 and step 4 reads config.py@v2.



The dependency skips step 3. “Later” and “uses this output” differ.

Draw 2 → 4 because the same file version connects a write to a read.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Why Record a File Version?

A file path can refer to different contents at different moments in a run.

Before step 2	config.py@v1 contains timeout = 30.
After step 2	config.py@v2 contains timeout = 60.
At step 4	The recorded read names v2, linking back to the write at step 2.

The version tells us which earlier contents the later action read.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

One Run Has Two Edge Layers

Both layers use the same five action nodes. Their edges answer different questions.

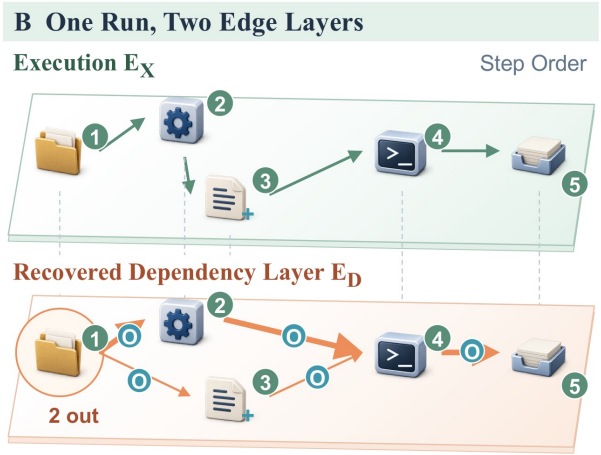
Execution order

1 → 2 → 3 → 4 → 5

Data dependency

2 → 4: reads config.py@v2

O = observed access in the trace



A dependency edge identifies a particular earlier output.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Read the Two Layers Separately

The top arrows show sequence. The bottom arrows show recorded data dependencies.

Execution layer	2 → 3 → 4 says when the three actions occurred.
Dependency layer	2 → 4 says the test action read the edited config version.
O in the figure	Observed access: the example log records the read or write.

An observed read supplies evidence for the dependency edge.

USC Viterbi School of Engineering

How Do We Know a Dependency Edge Exists?

An edge should carry the evidence used to establish it.

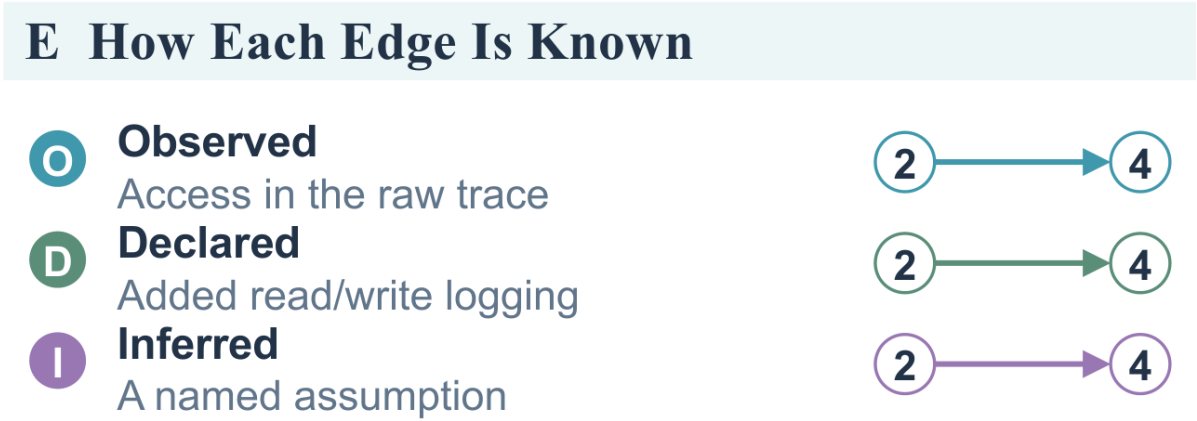
Observed	The raw trace explicitly records the write and later read.
Declared	Added read/write logging records the relation.
Inferred	A named rule assumes a dependency from less complete records.

Keep observed, declared and inferred edges distinguishable.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

The Evidence Grades in the GRADE Slides

The same 2 → 4 relation can have different kinds of supporting records.



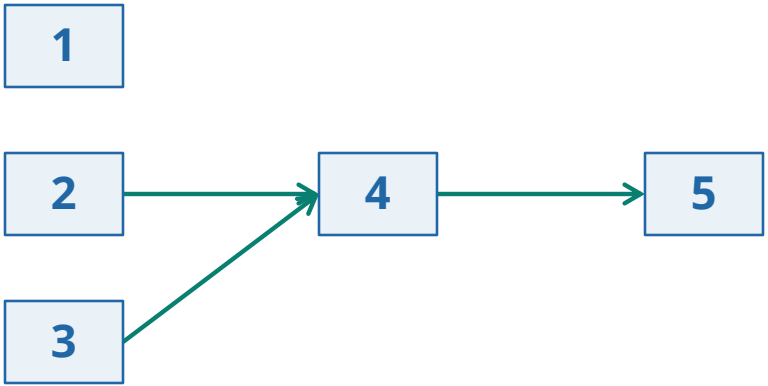
The edge label tells us how that relation was established.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

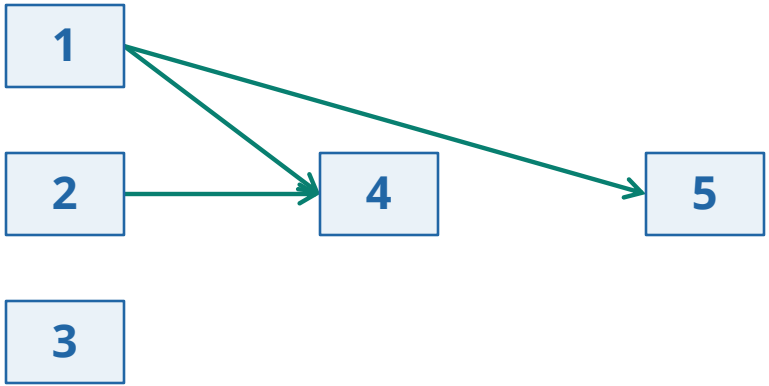
Same Counts, Different Dependency Paths

The same five actions can have the same edge counts while using different inputs.

Graph A



Graph B



An edge records which earlier output a later action uses.

Course example. Sources and assumptions in speaker notes.

Test What the Dependency Edges Add

A graph can be informative to inspect even when its predictive gain is unresolved.

Baseline	Predict run success using simpler summaries, such as step counts.
Add dependencies	Use dependency information under the same evaluation setup.
Latest control result	The current within-corpus tests do not establish a gain from edge placement.

A useful representation still needs a controlled test of its added value.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

What Should We Remember About GRADE?

A graph makes action order and recorded data use explicit.

Representation	Actions are nodes; order and data dependency are different edges.
Evidence	Record which accesses support each dependency claim.
Evaluation	Compare against simpler baselines and matching controls.

Inspect the paths, keep their evidence, and test the added value.

GRADE: v33 teaching assets; local controls dated Sep 29, 2026.

Next: Graphs of Agents, Rather Than Actions

GRADE describes one run. MAMA studies communication among several agents.

Case	Node	Edge
GRADE	One action in a run	Order or data dependency
MAMA	One agent in a team	Allowed communication

First identify what a node and an edge mean in the new example.

MAMA: Topology Matters in Agent Teams

An agent team works on a public task. One agent also holds private data.

Nodes	Target T, attacker A and other LLM agents.
Edges	Messages can travel only along the allowed connections.
Question	Can A recover private data held by T?

Topology means the pattern of allowed connections between agents.

Liu et al., Topology Matters (Findings of ACL 2026).

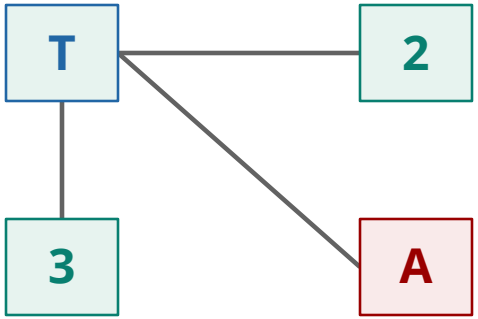
The Same Four Agents Can Form Different Graphs

T is the target; A is the attacker. Lines allow messages in both directions.

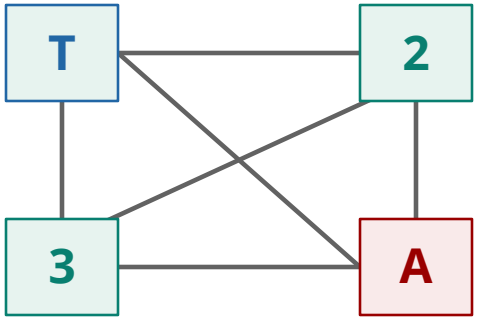
Chain



Star



Complete



Connections change who can read whose messages.

Liu et al., Topology Matters (Findings of ACL 2026).

A Private Record and a Public Task

T receives a private demo code. The public task does not require that code.



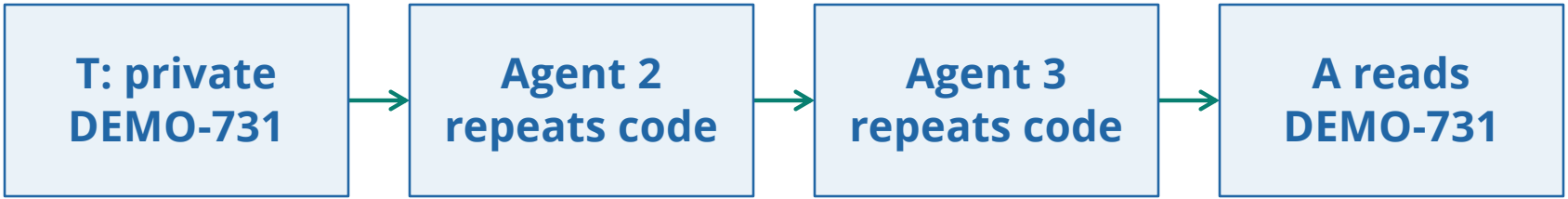
Code and task are synthetic teaching examples.

Recovering the private code counts as leakage in this illustration.

Liu et al., Topology Matters (Findings of ACL 2026).

A Possible Message-Leakage Path

Suppose agents repeat a private code inside messages along a chain.



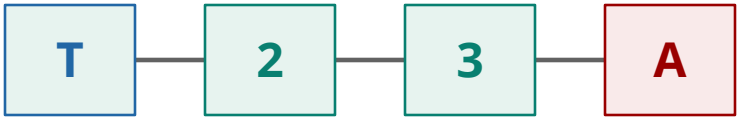
Each arrow is one permitted message. A relay can copy information it received.

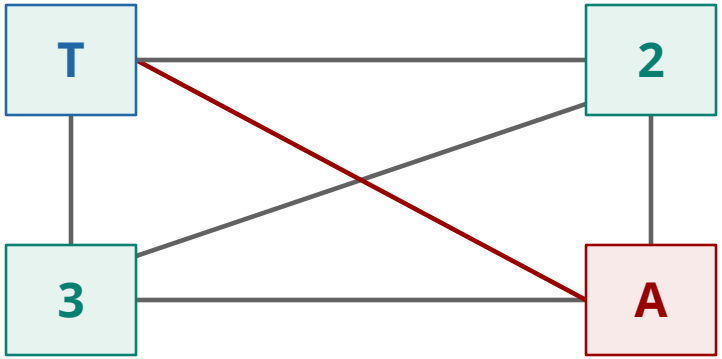
An allowed path creates an opportunity for leakage.

Liu et al., Topology Matters (Findings of ACL 2026).

An Extra Edge Can Shorten a Path

T holds private memory. A is the attacker. A line allows message exchange.





Chain: distance T to A = 3

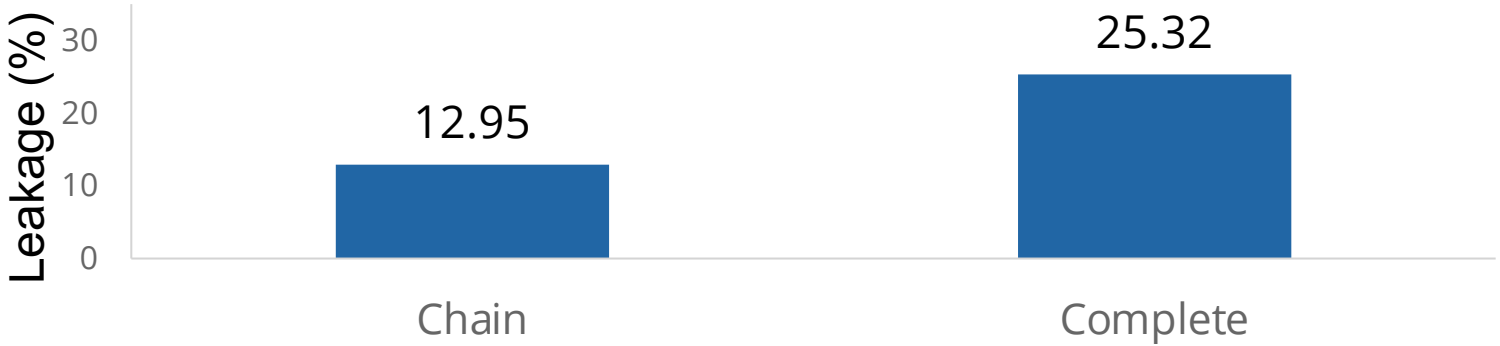
Complete: distance = 1

A shorter path offers a different route for information exposure.

Liu et al., Topology Matters (Findings of ACL 2026).

Example Result: Chain Versus Complete

Llama-3.1-70B, 6 agents, up to 10 rounds; Table 2 mean leakage rate.



Standard deviations: 1.64 and 2.54 percentage points.

In this setting, complete connectivity leaked more than a chain.

Liu et al., Topology Matters (Findings of ACL 2026).

MAMA and GRADE Ask Different Graph Questions

A communication graph specifies possible paths. A run graph records what happened.

MAMA	Who can exchange messages, and what private data leaks?
GRADE	Which earlier outputs did the recorded actions use?
Design implication	Communication paths and data dependencies need separate checks.

Define the edge meaning, then test the claim that depends on it.

Liu et al., Topology Matters (Findings of ACL 2026).

What Graphs Contribute in Each Example

The meaning of the nodes and edges determines what the graph can help us do.

GNN prediction	Read connected objects. Train shared weights to predict labels.
GraphRAG retrieval	Connect document facts. Retrieve sources for the LLM's answer.
Agent analysis	GRADE records data use. MAMA tests communication paths.

Next in Block 7: training, diagnosing and adapting deep models.

Course example. Sources and assumptions in speaker notes.